



ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

IT4490 – Thiết kế và xây dựng phần mềm

Bài 8. Kiểm thử đơn vị

1

Nội dung

- ➔ 1. Tổng quan về kiểm thử
2. Kiểm thử đơn vị
3. Kiểm thử tích hợp



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

2

Kiểm thử (Testing)

- ❖ “[T]he means by which the presence, quality, or genuineness of anything is determined; a means of trial.” –[dictionary.com](https://www.dictionary.com)
- ❖ **Kiểm thử phần mềm** thực thi một chương trình để xác định xem một thuộc tính của chương trình có đạt hay không
- ❖ Một test **vượt qua / passes** [hoặc **thất bại / fails**] nếu thuộc tính **đạt được / holds** [hoặc **không đạt được / doesn't hold**] trong lần chạy đó



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

3

Software Quality Assurance (QA)

Kiểm thử kết hợp với các hoạt động khác bao gồm

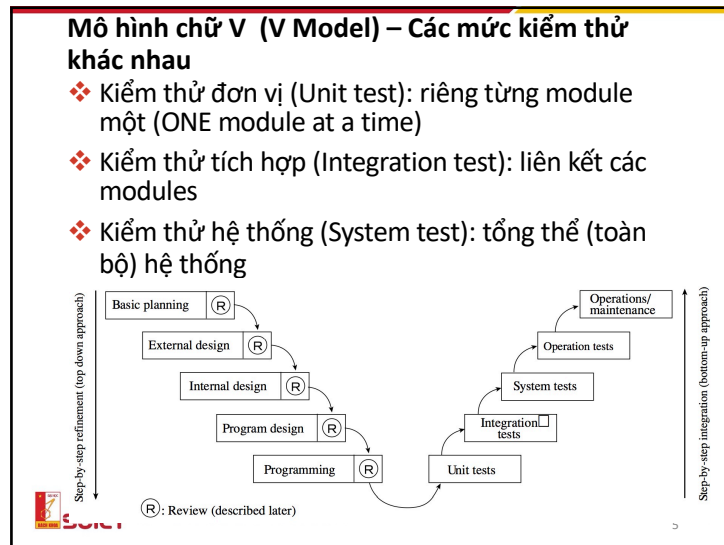
- ❖ Phân tích tĩnh (đánh giá mã nguồn mà không cần phải thực thi chúng)
- ❖ Chứng minh tính đúng đắn (các định lý về những thuộc tính của chương trình)
- ❖ Xét duyệt mã nguồn (mỗi người xét duyệt mã nguồn của những người khác)
- ❖ Quy trình phần mềm (đặt cấu trúc vào vòng đời phát triển)
- ❖ ... và nhiều cách khác để tìm ra vấn đề và tăng cường sự tự tin

**No single activity or approach
can guarantee software quality**



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

4



5

Các mức kiểm thử (Test levels)

- ❖ **Kiểm thử đơn vị (Unit Testing):** Mỗi đơn vị (lớp, phương thức, v.v.) có làm những gì nó phải làm không?
 - Đơn vị lập trình nhỏ nhất
 - Chiến lược: Kiểm thử hộp đen và kiểm thử hộp trắng
 - Các kỹ thuật, các công cụ
- ❖ **Kiểm thử tích hợp (Integration Testing):** bạn có nhận được kết quả mong đợi khi các bộ phận được kết hợp với nhau?
 - Các chiến lược: Kiểm thử từ dưới lên (Bottom-up), kiểm thử từ trên xuống (top-down)
- ❖ **Kiểm thử hệ thống (System Testing):** hệ thống tổng thể có hoạt động không?
- ❖ **Kiểm thử chấp nhận (Acceptance Testing):** nó có phù hợp với các yêu cầu của người dùng?

6

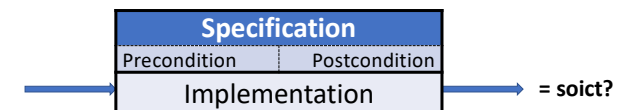
Nội dung

1. Tổng quan về kiểm thử
- ➔ 2. Kiểm thử đơn vị
3. Kiểm thử tích hợp

7

2.1. Các phương pháp tiếp cận kiểm thử đơn vị Kiểm thử hộp đen và hộp trắng

- A. Chọn dữ liệu đầu vào ("test inputs")
- B. Định nghĩa đầu ra mong đợi ("soict")
- C. Thực thi đơn vị ("SUT" or "software under test") trên đầu vào và ghi nhận các kết quả
- D. Kiểm tra kết quả có trái với đầu ra mong đợi ("soict")



Hộp đen (Black box)

Phải chọn đầu vào mà *không* có kiến thức về việc thực thi

Hộp trắng (White box)

Có thể chọn đầu vào với kiến thức về việc thực thi

8

Không chỉ black-and-white, mà...

Black box
Must choose inputs *without knowledge* of the implementation

- ❖ Phải tập trung vào hành vi của SUT
- ❖ Cần một “soic”
 - Hoặc ít nhất là một kỳ vọng về việc liệu **một ngoại lệ** có được ném ra hay không

White box
Can choose inputs *with knowledge* of the implementation

- ❖ Sử dụng phổ biến: bao phủ (**coverage**)
- ❖ Ý tưởng cơ bản: nếu bộ kiểm thử (test suite) của bạn khiến một câu lệnh không bao giờ được thực thi, thì câu lệnh đó có thể bị lỗi

 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG 9

9

Các thuật ngữ

Test Plan = **Test Strategy** + **Test Logistics**

↓

Test Strategy is Approach
i.e.
What, Why, When
& How
in Testing

↓

Test Logistics –
Who in Testing?

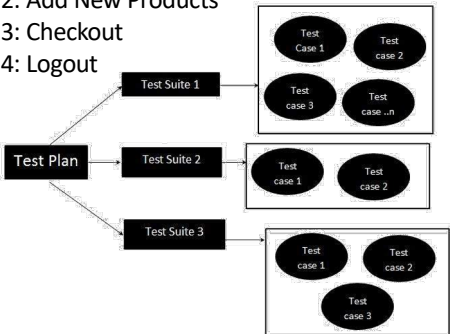
- ❖ Trường hợp kiểm thử (Test case)
 - một tập hợp các điều kiện / biến để xác định xem một hệ thống đang được kiểm thử có đáp ứng các yêu cầu hoặc hoạt động chính xác hay không
- ❖ Bộ kiểm thử (Test suite)
 - một tập hợp các trường hợp thử nghiệm liên quan đến cùng một công việc thử nghiệm
- ❖ Kế hoạch kiểm thử (Test plan)
 - tài liệu mô tả cách tiếp cận kiểm thử và phương pháp luận đang được sử dụng để thử nghiệm dự án, rủi ro, phạm vi kiểm thử, các công cụ cụ thể

 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG 10

10

Bộ kiểm thử (Test suite)

- ❖ Ví dụ về test suite
 - Test case 1: Login
 - Test case 2: Add New Products
 - Test case 3: Checkout
 - Test case 4: Logout



 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG 11

11

Các kỹ thuật kiểm thử đơn vị

- ❖ Dành cho thiết kế trường hợp thử
- ❖ (2.2) Các kỹ thuật kiểm thử cho kiểm thử hộp đen (Black Box Test)
 - Phân tích phân vùng tương đương (Equivalence Partitioning Analysis)
 - Phân tích giá trị biên (Boundary-value Analysis)
 - Bảng quyết định (Decision Table)
 - Kiểm thử dựa trên ca sử dụng (Use Case-based Test)
- ❖ (2.3) Các kỹ thuật kiểm thử cho kiểm thử hộp trắng (White Box Test)
 - Kiểm thử luồng điều khiển với phủ C0, C1 (Control Flow Test with C0, C1 coverage)
 - Kiểm thử phủ biểu đồ tuần tự (Sequence chart coverage test)

 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG 12

12

2.2. Các kỹ thuật kiểm thử hộp đen

2.2.1. Phân vùng tương đương

- ❖ Tạo các trường hợp thử nghiệm bao trùm bằng cách phân tích không gian dữ liệu đầu vào và chia thành các lớp tương đương
 - Không gian điều kiện đầu vào được phân chia thành các lớp tương đương
 - Mọi đầu vào được lấy từ một lớp tương đương tạo ra cùng một kết quả



13

Ví dụ: Chương trình Đánh giá Kỳ thi

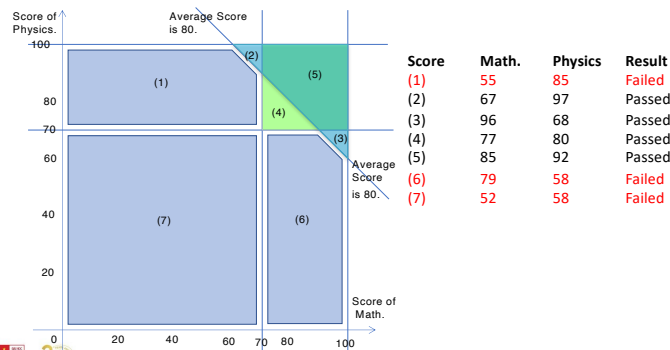
- ❖ Tên chương trình: "Chương trình đánh giá kỳ thi"
- ❖ Môn học: Hai môn Toán và Vật lý
- ❖ Đặc tả:
 - Đổ kỳ thi nếu
 - điểm của cả toán và vật lý lớn hơn hoặc bằng 70 trên 100
 - hoặc,
 - điểm trung bình của toán học và vật lý lớn hơn hoặc bằng 80 trên 100
 - Trượt => nếu ngược lại



14

Phân vùng tương đương của không gian đầu vào và các trường hợp kiểm thử

- ❖ 7 lớp tương đương => ít nhất 7 trường hợp kiểm thử / dữ liệu



15

Phân vùng tương đương Thảo luận và phân tích bổ sung

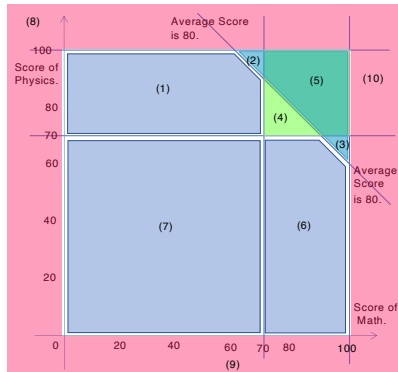
- ❖ Chúng ta có thành công không?
 - Không, chúng ta không thành công! Tại sao?
 - Còn thiếu một thứ!
- ❖ Phạm vi không gian đầu vào được phân tích là không đủ!
- ❖ Chúng ta phải thêm "Giá trị không hợp lệ" làm dữ liệu kiểm thử.
 - Ví dụ: một số mẫu "Giá trị không hợp lệ".
 - (8) Toán = -15, Vật lý = 120 Cả hai điểm đều không hợp lệ.
 - (9) Toán = 68, Vật lý = -66 Điểm Vật lý không hợp lệ.
 - (10) Toán = 118, Vật lý = 85 Điểm Toán không hợp lệ.



16

Thêm các lớp tương đương

❖ Thêm 3 test cases/data



Một số dữ liệu không hợp lệ đã được thêm.

Score	Math.	Physics	Result
(1)	55	85	Failed
(2)	67	97	Passed
(3)	96	68	Passed
(4)	77	80	Passed
(5)	85	92	Passed
(6)	79	58	Failed
(7)	52	58	Failed
(8)	-15	120	Invalid
(9)	68	-66	Invalid
(10)	118	85	Invalid

17

Phân tích và thảo luận

❖ Chúng ta đã cố gắng tạo các trường hợp thử nghiệm bao gồm dựa trên đặc điểm kỹ thuật bên ngoài.

- Thành công? "Đúng"!

❖ Câu hỏi tiếp theo. Các trường hợp / dữ liệu thử nghiệm có đầy đủ hiệu quả?

- Chúng ta phải tập trung vào nơi còn nhiều khiếm khuyết, phải không?
- Chỗ đó là ở đâu?

→ "Phân tích giá trị biên"

18

2.2. Các kỹ thuật kiểm thử hộp đen

2.2.2. Phân tích giá trị biên

❖ Trích xuất dữ liệu kiểm tra được mong đợi bằng cách phân tích các giá trị đầu vào biên => Dữ liệu kiểm tra hiệu quả

- Giá trị biên có thể phát hiện nhiều khiếm khuyết một cách hiệu quả

→ Ví dụ mathematics/physics score bằng 69 và 70

- Lập trình viên đã mô tả đoạn mã sai như sau:

```
if (mathscore > 70) {
    .....
}
```

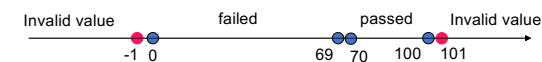
- Thay vì viết mã đúng như sau;

```
if (mathscore >= 70) {
    .....
}
```

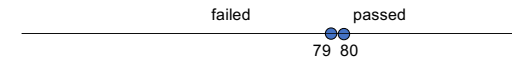
19

Ví dụ: Phân tích giá trị biên

❖ Các giá trị biên của điểm toán trong case study:



❖ Còn về phân tích giá trị biên cho điểm trung bình của toán và vật lý?



20

2.2. Các kỹ thuật kiểm thử hộp đen

2.2.3. Bảng quyết định

- ❖ Mỗi quan hệ giữa các điều kiện và nội dung của quá trình xử lý được thể hiện dưới dạng một bảng
- ❖ Bảng quyết định là một công cụ dạng bảng được sử dụng khi các điều kiện phức tạp được kết hợp

21

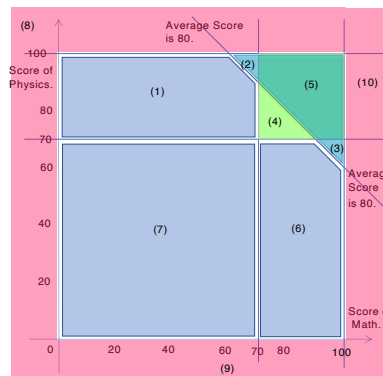
Ví dụ: Bảng quyết định

- ❖ Các điều kiện để tạo báo cáo từ tệp nhân viên

Under age 30	Y	Y	N	N
Male	Y	N	Y	N
Married	N	Y	Y	N
Output Report 1	-	X	-	-
Output Report 2	-	-	-	X
Output Report 3	X	-	-	-
Output Report 4	-	-	X	-

22

Bảng quyết định cho “Examination Judgement”???



Test Data from Equivalence Analysis

Score	Math.	Physics	Result
(1)	55	85	Failed
(2)	67	97	Passed
(3)	96	68	Passed
(4)	77	80	Passed
(5)	85	92	Passed
(6)	79	58	Failed
(7)	52	58	Failed
(8)	15	120	Invalid
(9)	68	-66	Invalid
(10)	118	85	Invalid

23

Bảng quyết định cho “Examination Judgement”

Condition1: Mathematics score>=>70

Condition2: Physics score=>70

Condition3: Average of Mathematics, and Physics =>80

	TC5	TC4	TC3	TC6	TC2	TC1	TCNG	TC7
Condition1	True	True	True	True	False	False	False	False
Condition2	True	True	False	False	True	True	False	False
Condition3	True	False	True	False	True	False	True(none)	False
"Passed"	Yes	Yes	Yes	---	Yes	---	N/A	--
"Failed"	---	---	---	Yes	---	Yes	N/A	Yes

24

Bảng quyết định cho “Examination Judgement”

❖ Dữ liệu vào không hợp lệ (integer)

- Condition1: Mathematics score = valid that means “0<= the score <= 100”
- Condition2: Physics score = valid that means “0<= the score <= 100”

	TC1	TC2	TC3	TC4
Condition1	Valid Invalid	Valid	Invalid	
Condition2	Valid Valid	Invalid	Invalid	
“Normal results”	Yes	---	---	---
“Error message math”	---	Yes	---	Yes
“Error message phys”	---	---	Yes	Yes

Nếu cả điểm toán và điểm vật lý đều không hợp lệ, hai thông báo sẽ được xuất ra. Đây có phải là một đặc tả chính xác không? Hãy xác nhận nó?

25

2.2. Các kỹ thuật kiểm thử hộp đen

2.2.4. Kiểm thử cho Use case

❖ ???

❖ Ví dụ. Bảng quyết định cho Login

- Các điều kiện
 - ???
- Các kết quả
 - ???

❖ Ví dụ. Phân tích giá trị biên

- ?

26

Các trường hợp kiểm thử cho “Log in”

❖ “Thành công”

- Mã PIN đúng

❖ “Thất bại”

- Mã PIN sai và số lần sai < 3

❖ “Khoá tài khoản”

- Mã PIN sai và số lần sai = 3

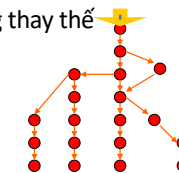
Mã PIN đúng	Y	Y	N	N
Số lần sai < 3	Y	N	Y	N
“Thành công”	x	N/A	-	-
“Thất bại”	-	N/A	x	-
“Khoá tài khoản”	-	N/A	-	x

- Phân tích vùng biên? Số lần sai = 2, 4 (?)

27

Tạo các trường hợp kiểm thử từ các ca sử dụng

- Xác định tất cả các kịch bản cho trường hợp sử dụng nhất định
- Các kịch bản thay thế nên được vẽ dưới dạng biểu đồ cho mỗi hành động
- Tạo kịch bản cho
 - a basic flow,
 - một kịch bản cho mỗi luồng thay thế,
 - và một số kết hợp hợp lý của các luồng thay thế
- Tạo vòng lặp vô hạn



28

2.3. Các kỹ thuật kiểm thử hộp trắng

❖ Các trường hợp kiểm thử phải bao gồm tất cả cấu trúc xử lý trong mã nguồn

=> Phủ kiểm thử điển hình

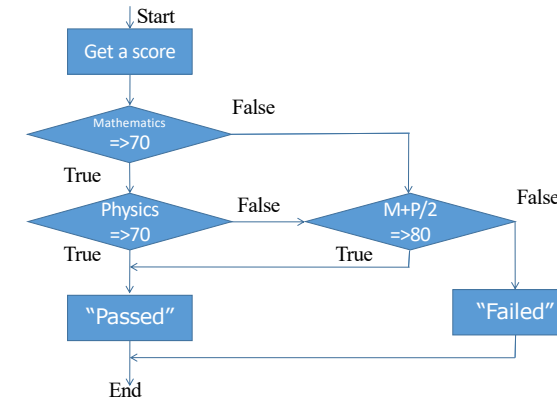
- C0 measure: Các câu lệnh đã thực thi # / tất cả các câu lệnh #
 - C0 đo ở mức 100% có nghĩa là "tất cả các câu lệnh được thực hiện"
- C1 measure: Các nhánh vượt qua # / tất cả các nhánh #
 - C1 đo ở mức 100% có nghĩa là "tất cả các nhánh được thực hiện"

=> Ngăn các câu lệnh / nhánh không được để lại như các phần không được kiểm tra

=> Không thể phát hiện các chức năng không được triển khai

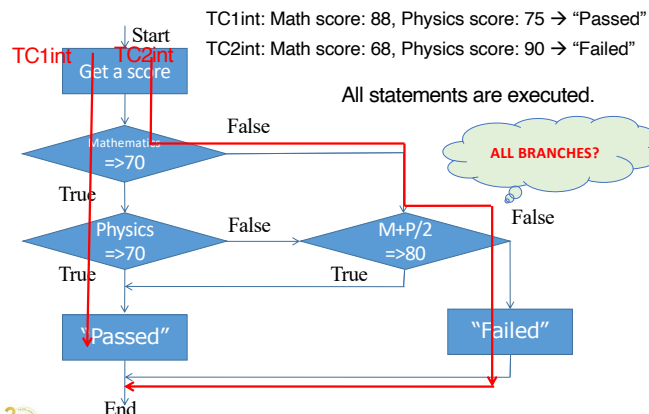
29

VD. Kiểm thử luồng điều khiển cho "Examination Judgment Program"



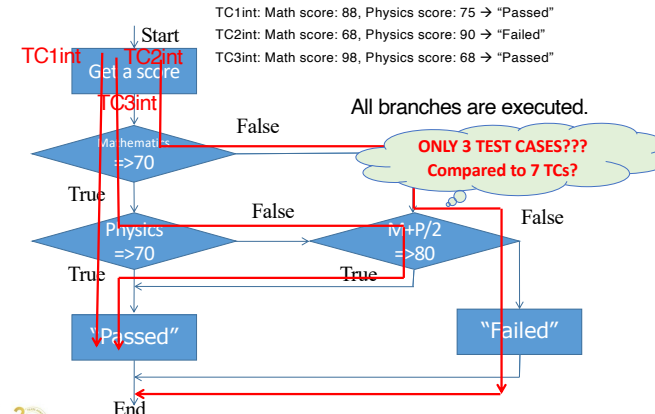
30

VD. Kiểm thử luồng điều khiển cho "Examination Judgment Program" – 100% C0 coverage



31

VD. Kiểm thử luồng điều khiển cho "Examination Judgment Program" – 100% C1 coverage



32

Bảng quyết định cho “Examination Judgment”

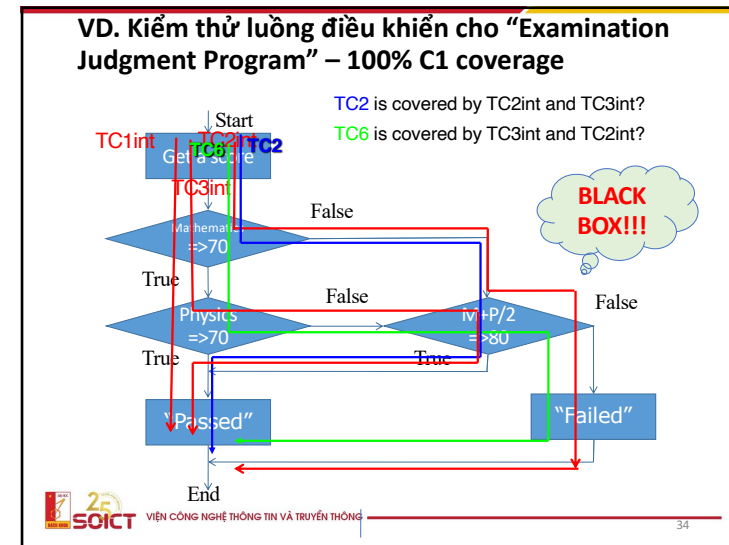
Condition1: Mathematics score $\Rightarrow$ 70
 Condition2: Physics score $\Rightarrow$ 70
 Condition3: Average of Mathematics, and Physics $\Rightarrow$ 80

	TC5	TC4	TC3	TC6	TC2	TC1	TCNG	TC7	
Condition1	True	True	True	True	False	False	False	False	False
Condition2	True	True	False	False	True	True	False	False	False
Condition3	True	False	True	False	True	False	True (none)	False	False
"Passed"	Yes	Yes	Yes	—	Yes	—	N/A	—	—
"Failed"	—	—	—	Yes	—	Yes	N/A	Yes	Yes

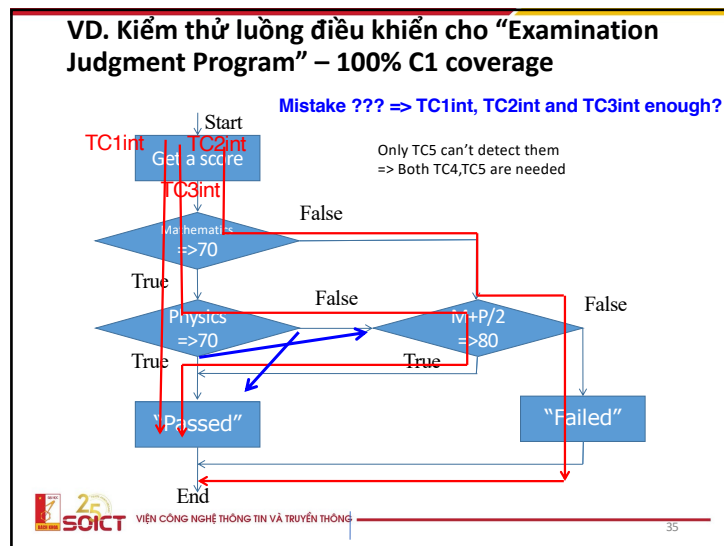
- ❖ One TCxint can cover plural TCs, based on the correct control flow structure
 - TC1int covers TC5 and TC4
 - TC2int covers TC1 and TC7
 - TC3int covers to TC3.
- ❖ TC2 and TC6 are left in no execution

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

33



34



35

Kiểm thử đường dẫn dữ liệu / thông điệp để kiểm tra tích hợp

- ❖ Thực hiện kiểm thử hộp trắng bằng cách sử dụng biểu đồ tuần tự để kiểm tra tích hợp.
- $\Rightarrow$ Thực thi mọi đường dẫn / luồng thông điệp
- $\Rightarrow$ 100% message path/flow coverage
- ❖ Có thể áp dụng cho dữ liệu / đường dẫn thông điệp / lưu đồ hoặc sơ đồ khác

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

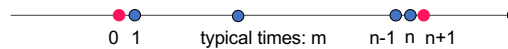
36

Cách kiểm tra chương trình cấu trúc vòng lặp

❖ Đối với kiểm tra luồng điều khiển trong phần mềm bao gồm một vòng lặp, các tiêu chí sau thường được áp dụng thay vì các độ đo bao phủ C0 / C1..

- Skip the loop.
- Only one pass through the loop.
- Typical times m passes through the loop
- n, n-1, n+1 passes through the loop
 - n is maximum number, m is typical number ($m < n$)

❖ Ví dụ: 6 trường hợp dựa trên phân tích giá trị biên:



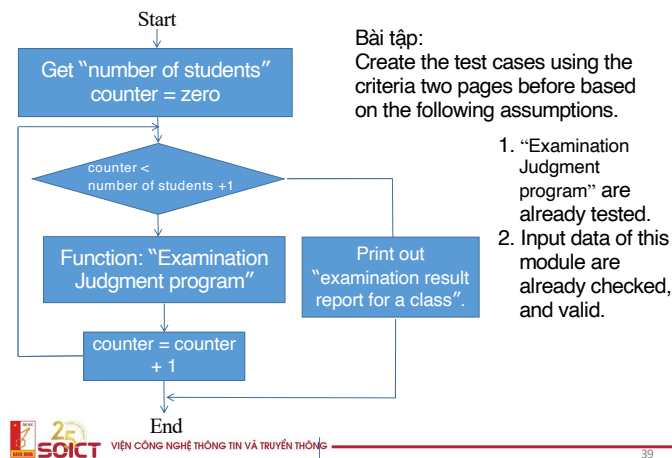
37

Các ví dụ cho "Examination Judgment Program"

- Nhập điểm hai môn Toán và Vật lý cho mỗi thành viên của một lớp.
 - input form là dạng "tabular form".
 - Các thành viên trong lớp chỉ có thể được phép từ 0 (không) đến 50.
- Xuất / In ra "Báo cáo kết quả thi của một lớp".
 - Biểu mẫu đầu ra cũng là "dạng bảng" có các cột như tên học sinh, điểm số (Toán, Lý), đạt hay không đạt.

38

Các ví dụ cho "Examination Judgment Program"

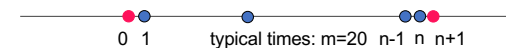


39

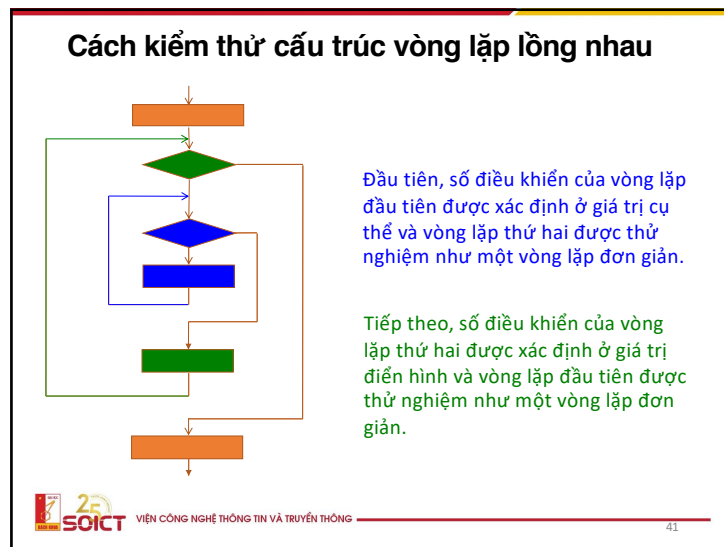
Các ví dụ cho "Examination Judgment Program"

Loop test cases of the module are; $n = 50$.

- "number of students" = 0,
- "number of students" = 1,
- "number of students" = 20,
- "number of students" = 49,
- "number of students" = 50,
- "number of students" = 51 → Invalid.



40



41

2.4. Kết hợp Black/White Box test

- ❖ Ưu điểm của Black box
 - Kiểm tra bao gồm dựa trên đặc điểm kỹ thuật bên ngoài
 - Rất mạnh mẽ và cơ bản để phát triển phần mềm chất lượng cao
- ❖ Ưu điểm của White box
 - Nếu bất kỳ đường dẫn / luồng nào không xuất hiện trong các thông số kỹ thuật đã viết, các đường dẫn / luồng có thể bị bỏ sót trong các bài kiểm thử bao gồm => White box test
 - cho dữ liệu của hơn hai năm trước => đường dẫn thay thế
 - "0 <= score <= 100" => code: "if 0 <= score" and "if score <= 100"

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

42

Cách thực hiện kiểm tra hiệu quả và đầy đủ

- ❖ Đầu tiên, thực hiện các bài kiểm tra dựa trên các thông số kỹ thuật bên ngoài
 - Nếu tất cả các trường hợp thử nghiệm đều thành công
 - => Tất cả các thông số kỹ thuật bên ngoài được thực hiện chính xác
- ❖ Thứ hai, thực hiện các bài kiểm tra dựa trên các thông số kỹ thuật bên trong
 - Thêm các trường hợp thử nghiệm để thực thi các đường dẫn / luồng còn lại, trong các thông số kỹ thuật bên ngoài
 - Nếu tất cả các trường hợp thử nghiệm đều thành công với mức độ phù hợp = 100%
 - => Tất cả các chức năng được chỉ định trong thông số kỹ thuật bên ngoài được thực hiện thành công mà không có bất kỳ mã dư thừa nào

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

43

2.5. JUnit

- ❖ Một công cụ test-driven development (junit.org)
- ❖ JUnit test generators hiện là một phần của nhiều Java IDEs (Eclipse, BlueJ, Jbuilder, DrJava)
- ❖ Các công cụ XUnit kể từ đó đã được phát triển cho nhiều ngôn ngữ khác (Perl, C ++, Python, Visual Basic, C #,...)

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

44

Tại sao phải tạo một bộ thử nghiệm?

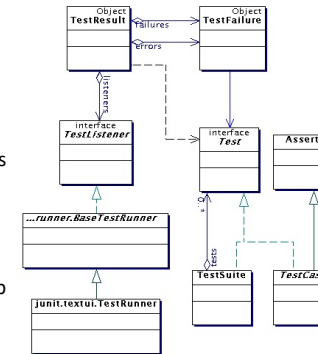
- ❖ Rõ ràng là bạn phải kiểm tra mã của mình — phải không?
 - Bạn có thể thực hiện thử nghiệm đột xuất (chạy bất kỳ thử nghiệm nào xảy ra với bạn vào lúc này) hoặc
 - Bạn có thể xây dựng một bộ thử nghiệm (một bộ thử nghiệm kỹ lưỡng có thể chạy bất cứ lúc nào)
- ❖ Nhược điểm của bộ thử nghiệm (test suite)
 - Phải lập trình thêm nhiều thứ
 - True, but use of a good test framework can help quite a bit
 - Bạn không có thời gian để làm thêm tất cả những công việc đó
 - False! Các thử nghiệm lặp đi lặp lại cho thấy rằng các bộ thử nghiệm giảm thời gian gỡ lỗi nhiều hơn so với số tiền dành để xây dựng bộ thử nghiệm
- ❖ Ưu điểm của bộ thử nghiệm
 - Giảm tổng số lỗi trong mã đã phân phối
 - Làm cho mã dễ bảo trì và dễ tái cấu trúc hơn nhiều



45

Tổng quan kiến trúc

- ❖ Khung kiểm thử JUnit là một gói các lớp cho phép bạn viết các bài kiểm thử cho từng phương thức, sau đó dễ dàng chạy các bài kiểm tra đó
- ❖ **TestRunner** thực thi các kiểm thử và xuất báo cáo **TestResults**
- ❖ Bạn kiểm thử một lớp bằng cách mở rộng lớp ch trừu tượng **TestCase**
- ❖ Để viết các trường hợp kiểm thử, bạn cần biết và hiểu về lớp **Assert** class



46

Viết một TestCase

- ❖ Để bắt đầu sử dụng JUnit, hãy tạo một lớp con của TestCase, lớp này bạn thêm các phương thức kiểm tra
- ❖ Đây là khung lớp kiểm thử:

```
import junit.framework.TestCase;
public class TestBowl extends TestCase {

    } //Test my class Bowl
```
- ❖ Tên của lớp rất quan trọng— nên có dạng là **TestMyClass** hoặc **MyClassTest**
- ❖ Quy ước đặt tên này giúp TestRunner tự động tìm các lớp kiểm thử của bạn



47

Viết các phương thức trong TestCase

- ❖ Mẫu dưới đây, mô thức lập trình theo hợp đồng **programming by contract** paradigm:
 - Thiết lập **preconditions**
 - Exercise functionality being tested
 - Kiểm tra **postconditions**
- ❖ Ví dụ:

```
public void testEmptyList() {
    Bowl emptyBowl = new Bowl();
    assertEquals("Size of an empty list should be zero.",
        0, emptyList.size());
    assertTrue("An empty bowl should report empty.",
        emptyBowl.isEmpty());
}
```
- ❖ Những điều cần lưu ý:
 - Chữ ký phương thức xác định — public void **testWhatever()**
 - Cho phép chúng được tìm thấy và gọi tự động bởi JUnit
 - Viết mã theo mẫu
 - Chú ý các lời gọi assert-type calls...



48

Các phương thức Assert

- ❖ Các phương thức Assert methods có các tham số dạng như sau:
message, expected-value, actual-value
- ❖ Floating point numbers get an additional argument, a tolerance
- ❖ Mỗi phương thức assert có một phiên bản tương đương không nhận thông báo - tuy nhiên, việc sử dụng này không được khuyến khích vì:
 - Các thông điệp giúp ghi nhận các kiểm thử
 - Các thông điệp cung cấp thông tin bổ sung khi đọc nhật ký lỗi

49

Các phương thức Assert

- ❖ `assertTrue(String message, Boolean test)`
- ❖ `assertFalse(String message, Boolean test)`
- ❖ `assertNull(String message, Object object)`
- ❖ `assertNotNull(String message, Object object)`
- ❖ `assertEquals(String message, Object expected, Object actual)`
(uses equals method)
- ❖ `assertSame(String message, Object expected, Object actual)`
(uses == operator)
- ❖ `assertNotSame(String message, Object expected, Object actual)`

50

Nhiều thứ hơn trong các lớp thử nghiệm

- ❖ Giả sử bạn muốn kiểm thử một lớp `Counter`
- ❖ `public class CounterTest`
`extends junit.framework.TestCase {`
 - This is the unit test for the `Counter` class
- ❖ `public CounterTest() { } //Default constructor`
- ❖ `protected void setUp()`
 - Test *fixture* creates and initializes instance variables, etc.
- ❖ `protected void tearDown()`
 - Releases any system resources used by the test fixture
- ❖ `public void testIncrement(), public void testDecrement()`
 - These methods contain tests for the `Counter` methods `increment()`, `decrement()`, etc.
 - Note capitalization convention

51

JUnit tests for Counter

```
public class CounterTest extends junit.framework.TestCase {
    Counter counter1;
    public CounterTest() { } // default constructor

    protected void setUp() { // creates a (simple) test fixture
        counter1 = new Counter();
    }

    public void testIncrement() {
        assertTrue(counter1.increment() == 1);
        assertTrue(counter1.increment() == 2);
    }

    public void testDecrement() {
        assertTrue(counter1.decrement() == -1);
    }
}
```

Lưu ý rằng mỗi bài kiểm thử bắt đầu với một bộ đếm hoàn toàn mới

Điểm này có nghĩa là bạn không phải lo lắng về thứ tự chạy các bài kiểm thử

52

TestSuites

- ❖ TestSuites tập hợp các bài kiểm thử đã lựa chọn để chạy chúng như một đơn vị
- ❖ Các bộ sưu tập tự động sử dụng TestSuites, tuy nhiên, để chỉ định thứ tự chạy các bài kiểm thử, hãy viết :

```
public static Test suite() {
    suite.addTest(new TestBowl("testBowl"));
    suite.addTest(new TestBowl("testAdding"));
    return suite;
}
```

- ❖ Ít khi phải viết TestSuites của riêng bạn vì mỗi phương thức trong TestCase của bạn phải độc lập với tất cả các phương thức khác
- ❖ Can create TestSuites that test a whole package:

```
public static Test suite() {
    TestSuite suite = new TestSuite();
    suite.addTestSuite(TestBowl.class);
    suite.addTestSuite(TestFruit.class);
    return suite;
}
```

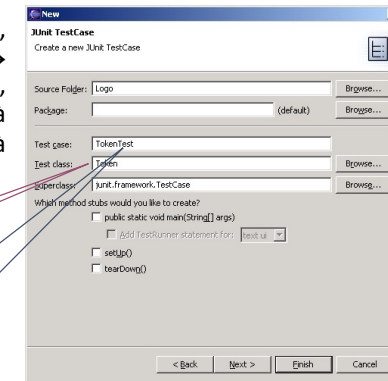
53

JUnit in Eclipse

- ❖ Để tạo một test mới, chọn File → New → Other... → Java, JUnit, TestCase và nhập tên của lớp mà bạn sẽ test

Fill this in

This will be filled in automatically



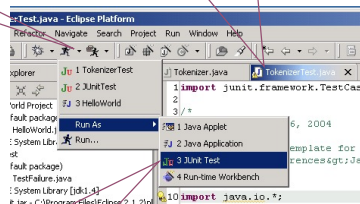
54

Thực thi JUnit

Second, use this
pull-down menu

First, select a Test class

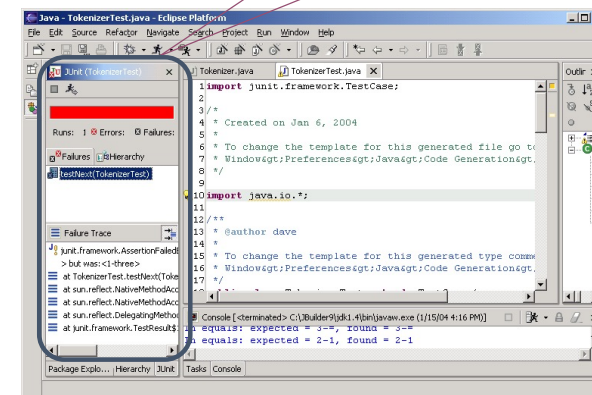
Third, Run As → JUnit Test



55

Kết quả

Your results are here



56

Kiểm thử đơn vị cho các ngôn ngữ khác

- ❖ Các công cụ kiểm thử đơn vị phân biệt giữa :
 - Errors (unanticipated problems caught by exceptions)
 - Failures (anticipated problems checked with assertions)
- ❖ Đơn vị cơ bản của kiểm thử:
 - `CPPUNIT_ASSERT(Bool)` kiểm tra một biểu thức
- ❖ CppUnit có nhiều lớp kiểm thử (e.g. `TestFixture`)
 - Kế thừa chúng và nạp chồng các phương thức

57

Một ví dụ khác: sqrt

```
// throws: IllegalArgumentException if x < 0
// returns: approximation to square root of x
public double sqrt(double x)
```

Một số giá trị hoặc phạm vi của x có thể đáng kiểm thử là gì

- ❑ $x < 0$ (exception thrown)
- ❑ $x \geq 0$ (returns normally)
- ❑ around $x = 0$ (boundary condition)
- ❑ perfect squares (`sqrt(x)` an integer), non-perfect squares
- ❑ $x < \text{sqrt}(x)$, $x > \text{sqrt}(x)$
- ❑ Specific tests: say $x = \{-1, 0, 0.5, 1, 4\}$

58

Các miền con

- ❖ Nhiều lần thực thi phản ánh cùng một hành vi - ví dụ: đối với `sqrt`, kỳ vọng là
 - Tất cả $x < 0$ đầu vào sẽ ném ra một ngoại lệ
 - Tất cả $x \geq 0$ đầu vào sẽ trả về bình thường với một câu trả lời đúng
- ❖ Bằng cách thử nghiệm bất kỳ phần tử nào từ mỗi **miền con**, mục đích là để thử nghiệm đơn lẻ đại diện cho các hành vi khác của tên miền phụ - *mà không cần thử nghiệm chúng!*
- ❖ Tất nhiên, điều này không dễ dàng như vậy - ngay cả trong ví dụ đơn giản ở trên, còn khi x tràn?

59

Kiểm thử RandomHello

- ❖ “Tạo lớp Java đầu tiên của bạn với phương thức main sẽ chọn giá trị ngẫu nhiên, rồi in ra console, một trong năm lời chào có thể có mà bạn định nghĩa.”
- ❖ Chúng ta sẽ tập trung vào phương thức `getGreeting`, trả về ngẫu nhiên một trong năm lời chào
- ❖ Chúng ta sẽ tập trung vào *black-box testing* – chúng tôi sẽ làm việc mà không có kiến thức về việc triển khai
- ❖ Và chúng ta sẽ tập trung vào kiểm thử đơn vị bằng cách sử dụng JUnit framework
- ❖ Trộn lẫn, với bất kỳ sự may mắn nào, trang trình bày và bản trình diễn

60

Có thực thi và trả về kết quả?

- ❖ Nếu `getGreeting` không chạy và trả về mà không ném ra một ngoại lệ, nó không thể đáp ứng đặc tả

JUnit tag "this is a test"	<code>@Test</code>
name of test	<code>public void test_NoException() {</code>
Run <code>getGreeting</code>	<code>RandomHello.getGreeting();</code>
JUnit "test passed" (doesn't execute if exception thrown)	<code>assertTrue(true);</code> <code>}</code>

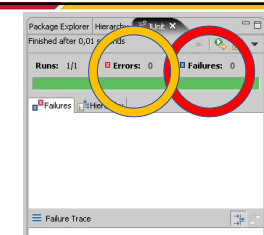
Tests should have descriptive (often very long) names

A unit test is a (stylized) program!
When you're writing unit tests (and many other tests), you're programming!

61

Thực thi JUnit tests

- ❖ Có nhiều cách để chạy phương thức kiểm thử JUnit, các lớp kiểm thử và bộ kiểm thử
- ❖ Tổng quát, chọn method, class hoặc suite và Run As >> JUnit Test
- ❖ Thanh màu xanh cho "all tests pass"
- ❖ Thanh màu đỏ cho ít nhất một test failed hoặc đã có error
- ❖ The failure trace cho biết test nào thất bại và tại sao



- A failure is when the test doesn't pass – that is, the oracle it computes is incorrect
- An error is when something goes wrong with the program that the test didn't check for (e.g., a null pointer exception)

62

Có trả lại một trong những lời chào không?

- ❖ Nếu không trả về một trong những lời chào đã xác định, nó không thể đáp ứng đặc tả

```
@Test
public void testDoes_getGreeting_returnDefinedGreeting() {
    String rg = RandomHello.getGreeting();
    for (String s : RandomHello.greetings) {
        if (rg.equals(s)) {
            assertTrue(true);
            return;
        }
    }
    fail("Returned greeting not in greetings array");
}
```

63

A JUnit test class

```
import org.junit.*;
import static org.junit.Assert.*;

public class RandomHelloTest() {
    @Test
    public void test_ReturnDefinedGreeting() {
        ...
    }
    @Test
    public void test_EveryGreetingReturned() {
        ...
    }
    ...
}
```

Don't forget that Eclipse can help you get the right import statements – use Organize Imports (Ctrl-Shift-O)

- ❑ All `@Test` methods run when the test class is run
- ❑ That is, a JUnit test class is a set of tests (methods) that share a (class) name

64

Có trả về một lời chào ngẫu nhiên?

```
@Test
public void testDoes_getGreetingNeverReturnSomeGreeting() {
    int greetingCount = RandomHello.greetings.length;
    int count[] = new int[greetingCount];
    for (int c = 0; c < greetingCount; c++)
        count[c] = 0;
    for (int i = 1; i < 100; i++) {
        String rs = RandomHello.getGreeting();
        for (int j = 0; j < greetingCount; j++)
            if (rs.equals(RandomHello.greetings[j]))
                count[j]++;
    }
    for (int j = 0; j < greetingCount; j++)
        if (count[j] == 0)
            fail(j+"th [0-4] greeting never returned");
    assertTrue(true);
}
```

Run it 100 times

If even one greeting is never returned, it's unlikely to be random (~1-0.8¹⁰⁰)

65

Còn về một nhà phát triển nhếch nhác?

```
if (randomGenerator.nextInt(2) == 0) {
    return (greetings[0]);
} else
    return (greetings[randomGenerator.nextInt(5)]);
```

- ❑ Lật đồng xu và chọn một lời chào ngẫu nhiên hoặc một lời chào cụ thể
- ❑ Trước đó "có phải là ngẫu nhiên không?" kiểm thử hầu như sẽ luôn vượt qua khi triển khai này
- ❑ Nhưng nó không đáp ứng đặc tả, vì nó không phải là một lựa chọn ngẫu nhiên

66

Thay vì thế: Sử dụng thống kê đơn giản

```
@Test
public void test_UniformGreetingDistribution() {
    // ...count frequencies of messages returned, as in
    // ...previous test (test_EveryGreetingReturned)

    float chiSquared = 0f;
    float expected = 20f;
    for (int i = 0; i < greetingCount; i++)
        chiSquared = chiSquared +
            ((count[i]-expected)*
             (count[i]-expected))
            /expected;
    if (chiSquared > 13.277) // df 4, pvalue .01
        fail("Too much variance");
}
```

67

A JUnit test suite

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;

@RunWith(Suite.class)
@Suite.SuiteClasses({
    RandomHelloTest.class,
    SleazyRandomHelloTest.class
})
public class AllTests {
    // this class remains completely
    // empty, being used only as a
    // holder for the above
    // annotations
}
```

- ❑ Xác định một bộ cho mỗi chương trình (hiện tại)
- ❑ The suite cho phép nhiều lớp kiểm thử— each of which has its own set of `@Test` methods – to be defined and run together
- ❑ Add `tc.class` to the `@Suite.SuiteClasses` annotation if you add a new test class named `tc`
- ❑ Vì vậy, bộ kiểm thử JUnit là một tập hợp các lớp kiểm thử (khiến nó trở thành một tập hợp các phương thức kiểm thử)

68

Các phương thức JUnit assertion

...khiến kiểm thử hiện tại không thành công...

<code>fail()</code>	<code>immediately</code>
<code>assertTrue(tst)</code>	<code>if tst is false</code>
<code>assertFalse(tst)</code>	<code>if test is true</code>
<code>assertEquals(expected, actual)</code>	<code>if expected does not equal actual</code>
<code>assertSame(expected, actual)</code>	<code>if expected != actual</code>
<code>assertNotSame(expected, actual)</code>	<code>if oracle == actual</code>
<code>assertNull(value)</code>	<code>if value is not null</code>
<code>assertNotNull(value)</code>	<code>if value is null</code>

- ❖ Có thể thêm thông báo lỗi: `assertNull("Ptr isn't null", value)`
- ❖ **expected** là giá trị tiên tri- hãy nhớ đây là tham số đầu tiên (ngoài cùng bên trái)
- ❖ Bảng trên chỉ mô tả khi nào thất bại - điều gì xảy ra nếu một khẳng định thành công? Bài kiểm tra có vượt qua không?

ArrayList: các ví dụ kiểm thử

```
@Test
public void testAddGet1() {
    ArrayList list = new
        ArrayList();
    list.add(42);
    list.add(-3);
    list.add(15);
    assertEquals(42, list.get(0));
    assertEquals(-3, list.get(1));
    assertEquals(15, list.get(2));
}
```

```
@Test
public void testIsEmpty() {
    ArrayList list = new
        ArrayList();
    assertTrue(list.isEmpty());
    list.add(123);
    assertFalse(list.isEmpty());
    list.remove(0);
    assertTrue(list.isEmpty());
}
```

- ❑ Khái niệm cấp cao: kiểm tra hành vi kết hợp
 - ❑ Có thể **add** hoạt động khi được gọi một lần, nhưng không hoạt động khi được gọi hai lần
 - ❑ Có thể **add** tự hoạt động, nhưng thất bại (hoặc gây ra lỗi) sau khi gọi **remove**

Một vài gợi ý: cấu trúc dữ liệu

- ❖ Cần phải truyền nhiều mảng? Sử dụng array literals

```
public void exampleMethod(int[] values) { ... }
...
exampleMethod(new int[] {1, 2, 3, 4});
exampleMethod(new int[] {5, 6, 7});
```

- ❖ Cần một **ArrayList** nhanh?

```
List<Integer> list = Arrays.asList(7, 4, -2, 3, 9, 18);
```

- ❖ Cần một set, queue, etc. nhanh? Many take a list

```
Set<Integer> list = new HashSet<Integer>()
    Arrays.asList(7, 4, -2, 9));
```

Một vài gợi ý chung

- ❖ Thử nghiệm từng thứ một trong mỗi phương thức thử nghiệm
 - 10 bài kiểm thử nhỏ tốt hơn nhiều so với một bài kiểm thử lớn
- ❖ Be stingy with **assert** statements
 - Lệnh **assert** đầu tiên thất bại sẽ dừng kiểm thử- không cung cấp thông tin về việc liệu **assert** sau này có thất bại hay không
- ❖ Be stingy with logic
 - Tránh **try/catch** - nếu nó được đưa ra một ngoại lệ, hãy sử dụng **expected=** ... Nếu không, hãy để JUnit bắt nó

Trường hợp kiểm thử nguy hiểm

❖ Thứ tự kiểm thử phụ thuộc

- Nếu chạy Kiểm thử A trước Kiểm thử B cho kết quả khác với việc chạy Kiểm thử B rồi đến Kiểm thử A, thì có thể có điều gì đó khó hiểu và cần được trình bày rõ ràng

❖ Trạng thái chia sẻ có thể thay đổi

- Tests A và B cả hai đều sử dụng một đối tượng được chia sẻ – nếu A breaks the object, điều gì xảy ra với B?
 - Đây là một dạng của lệnh kiểm thử phụ thuộc
 - We will explicitly talk about invariants over data representations and testing if the invariants are ever broken

73

More JUnit

❖ Hết giờ - không muốn đợi mãi để kiểm thử hoàn thành

❖ Kiểm thử các ngoại lệ

```
@Test(expected =  
    ArrayIndexOutOfBoundsException.class)  
public void testBadIndex() {  
    ArrayList list = new ArrayList();  
    list.get(4); // this should raise the exception  
} // and thus the test will pass
```

❖ Setup [teardown] – methods to run before [after] each test case method [test class] is called

74

Nội dung

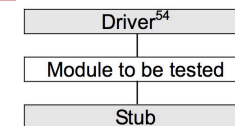
1. Tổng quan về kiểm thử

2. Kiểm thử đơn vị

➔ 3. Kiểm thử tích hợp

75

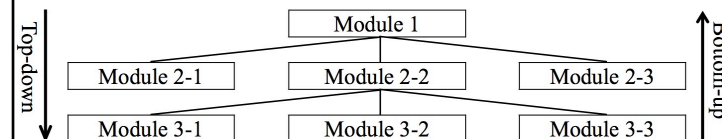
3. Kiểm thử tích hợp



❖ Kiểm tra giao diện giữa các mô-đun cũng như đầu vào và đầu ra

❖ Stub/Driver:

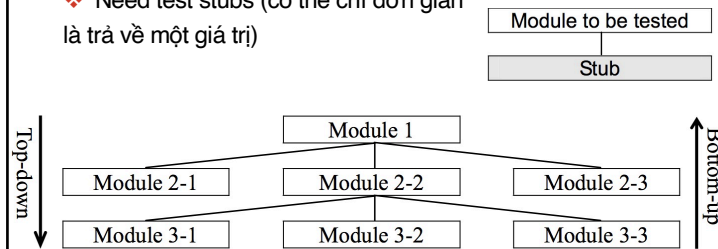
- Một chương trình mô phỏng các chức năng của mô-đun cấp thấp hơn / cấp cao hơn



76

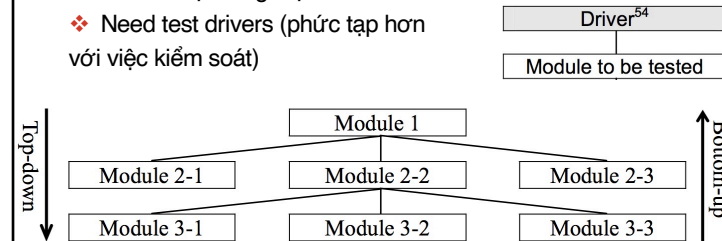
3.1. Cách tiếp cận từ trên xuống

- ❖ Có thể phát hiện sớm các khiếm khuyết do hiểu sai đặc tả
- ❖ Hiệu quả trong các hệ thống mới phát triển
- ❖ Need test stubs (có thể chỉ đơn giản là trả về một giá trị)



3.2. Cách tiếp cận từ dưới lên

- ❖ Các mô-đun thấp hơn là độc lập => kiểm tra độc lập và song song
- ❖ Hiệu quả trong việc phát triển hệ thống bằng cách sửa đổi các hệ thống hiện có
- ❖ Need test drivers (phức tạp hơn với việc kiểm soát)



3.3. Các kỹ thuật kiểm thử tích hợp khác

- ❖ Kiểm thử Big-bang
 - Trong đó tất cả các mô-đun đã hoàn thành các bài kiểm tra đơn vị được liên kết cùng một lúc và được kiểm tra
 - Giảm số lượng các thủ tục thử nghiệm trong chương trình quy mô nhỏ; nhưng không dễ xác định lỗi
- ❖ Kiểm thử Sandwich
 - Trong đó các mô-đun cấp thấp hơn được kiểm tra từ dưới lên và các mô-đun cấp cao hơn được kiểm tra từ trên xuống

3.4. Kiểm thử hồi quy

“Khi bạn sửa một lỗi, bạn sẽ tạo ra một số lỗi mới”

- ❖ Kiểm thử lại một ứng dụng sau khi mã của nó đã được sửa đổi để xác minh rằng nó vẫn hoạt động chính xác
 - Chạy lại các trường hợp kiểm thử hiện có
 - Kiểm tra để đảm bảo rằng các thay đổi mã không phá vỡ bất kỳ chức năng hoạt động nào trước đó (tác dụng phụ)(side-effect)
- ❖ Chạy thường xuyên nhất có thể
- ❖ Với công cụ kiểm thử hồi quy tự động



81



82


SOICT
 ĐẠI HỌC BÁCH KHOA HÀ NỘI
 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

IT4490 – Thiết kế và xây dựng phần mềm

Bài 9. Lập trình

1

For Your Amusement

- ❖ “Any fool can write code that a computer can understand. Good programmers write code that humans can understand” -- Martin Fowler
- ❖ “Good code is its own best documentation. As you’re about to add a comment, ask yourself, ‘How can I improve the code so that this comment isn’t needed?’” -- Steve McConnell
- ❖ “Programs must be written for people to read, and only incidentally for machines to execute.” -- Abelson / Sussman
- ❖ “Everything should be built top-down, except the first time.” -- Alan Perlis


SOICT
 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

2

Các cách để code của bạn đúng

- ❖ Xác minh / đảm bảo chất lượng
 - Mục đích là để khám phá các vấn đề và tăng cường sự tự tin
 - Kết hợp giữa lý luận và kiểm tra
- ❖ Debugging
 - Tìm hiểu lý do tại sao một chương trình không hoạt động như dự kiến
- ❖ Lập trình phòng thủ
 - Lập trình với xác nhận và gỡ lỗi trong tâm trí
- ❖ Testing ≠ debugging
 - test: tiết lộ tồn tại của vấn đề; bộ kiểm thử cũng có thể tăng độ tin cậy tổng thể
 - debug: xác định vị trí + nguyên nhân của sự cố


SOICT
 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

3

Nội dung

- ➡ 1. Phong cách lập trình
2. Tinh chỉnh / tối ưu mã (tuning / optimization)
3. Tái cấu trúc mã (refactoring)
4. Debugging


SOICT
 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

4

❖ Quy ước / tiêu chuẩn viết mã



General conventions: see <http://java.sun.com/doc/codecon/index.html>

```

package com.example;

import java.util.*;

public class HelloWorld implements InterfHello {

    public static final int MAX_SIZE = 123;

    static int height;

    /**
     * JavaDoc conventions: see
     * http://java.sun.com/doc/api/javahelp/creatingdoc.html
     *
     * @param the givenName, not getname() - one letters are OK
     * public String getname() { return getname(); }
     */

    public int getname() {
        if (height < MAX_SIZE) {
            return MAX_SIZE;
        }
        return height;
    }

    protected void printStr(String filename, Vector v) throws IOException {
        Writer out;
        try {
            out = new FileWriter(filename);
            output.println(filename);
            output.println(v.toString());
        } catch (FileNotFoundException fnf) {
            log.warn(fnf);
        } finally {
            close(out);
        }
        if (out != null) try {out.close();} catch(IOException ignored) {}
    }
}

```

Specify a package (not default)

package com.example;

import java.util.*;

May use ""

Each word capitalized

public class HelloWorld implements InterfHello {}

Each word capitalized non-prime + optional "able" or "ing"

All caps with "_" separator

public static final int MAX_SIZE = 123;

Constant

static int height;

No special prefix for "static" variables

/**

Noun phrase. Fields not "public"

JavaDoc conventions: see <http://java.sun.com/doc/api/javahelp/creatingdoc.html>

@param the givenName, not getname() - one letters are OK

public String getname() { return getname(); }

JavaBeans naming conventions: see <http://java.sun.com/beansdocs/beans101.pdf>

Braces here and here

4-space indent throughout

protected void printStr(String filename, Vector v) throws IOException {

Writer out;

try {

Exceptions either re-throw or handled

out = new FileWriter(filename);

output.println(filename);

output.println(v.toString());

} catch (FileNotFoundException fnf) {

log.warn(fnf);

"finally" clause to release resources

} finally {

close(out);

if (out != null) try {out.close();} catch(IOException ignored) {}

5

5

- ❖ Sử dụng hằng số cho tất cả các giá trị không đổi (ví dụ: thuế suất).
- ❖ Sử dụng các biến bất cứ khi nào có thể
- ❖ Luôn khai báo các hằng số trước các biến khi bắt đầu thủ tục
- ❖ Luôn chú thích mã nguồn (đặc biệt là các đoạn mã không rõ ràng hoặc gây hiểu lầm), nhưng không chú thích quá mức.
- ❖ Sử dụng tên mô tả thích hợp (có / không có tiền tố) cho các định danh / đối tượng (ví dụ: btnDone).



❖ Sử dụng dấu ngoặc cho các biểu thức toán học ngay cả khi không bắt buộc để có thể hiểu rõ ý định của nó.

- Ví dụ: $(3*2) + (4*2)$
- ❖ Luôn viết mã hiệu quả nhất có thể
- ❖ Tạo biểu mẫu input và output thân thiện với người dùng input and output



❖ Thêm một phần tiêu đề ở đầu mã nguồn:

- Tên người lập trình
- Ngày
- Tên của dự án đã lưu
- Tên của giáo viên
- Tên lớp
- Tên của bất kỳ ai đã giúp bạn
- Mô tả ngắn gọn về những gì chương trình thực hiện



Công cụ kiểm tra style

- ❖ Demo video

9

Nội dung

1. Phong cách lập trình
- ➔ 2. Tinh chỉnh / tối ưu mã (tuning / optimization)
3. Tái cấu trúc mã (refactoring)
4. Debugging

10

Tinh chỉnh mã (Code tuning)

- ❖ Sửa đổi mã chính xác để làm cho nó chạy hiệu quả hơn
- ❖ Không phải là cách hiệu quả nhất / rẻ nhất để cải thiện hiệu suất
- ❖ 20% các phương thức của chương trình tiêu tốn 80% thời gian thực thi của chương trình.

11

Code Tuning Myths

- ❖ Giảm các dòng mã bằng ngôn ngữ cấp cao sẽ cải thiện tốc độ hoặc kích thước của mã máy kết quả - sai!

```
for i = 1 to 10  
  a[ i ] = i  
end for
```

VS

```
a[ 1 ] = 1  
a[ 2 ] = 2  
a[ 3 ] = 3  
a[ 4 ] = 4  
a[ 5 ] = 5  
a[ 6 ] = 6  
a[ 7 ] = 7  
a[ 8 ] = 8  
a[ 9 ] = 9  
a[ 10 ] = 10
```

12

Code Tuning Myths (2)

- ❖ Một chương trình nhanh cũng quan trọng như một chương trình đúng - sai!



13

Code Tuning Myths (3)

- ❖ Một số hoạt động có thể nhanh hơn hoặc nhỏ hơn những hoạt động khác - sai!
 - Luôn đo lường hiệu suất!



14

Code Tuning Myths (4)

- ❖ Bạn nên tối ưu hóa ngay khi bạn viết mã - sai!
 - Thật khó để xác định các nút thắt cổ chai trước khi một chương trình hoạt động hoàn toàn
 - Tập trung vào tối ưu hóa làm giảm các mục tiêu chương trình khác

15

Khi nào thì tinh chỉnh

- ❖ Sử dụng thiết kế chất lượng cao
 - Thực hiện đúng chương trình.
 - Xây dựng theo mô-đun và dễ dàng sửa đổi
 - Khi hoàn thành và chính xác, hãy kiểm tra hiệu suất.
- ❖ Xem xét tối ưu hóa trình biên dịch
- ❖ Đo lường
- ❖ Viết mã rõ ràng, dễ hiểu và dễ sửa đổi.

16

Đo lường

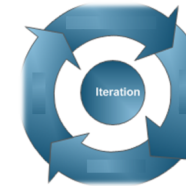
- ❖ Đo lường để tìm ra điểm nghẽn
- ❖ Các phép đo cần phải chính xác
- ❖ Các phép đo cần được lặp lại



17

Tối ưu hóa trong các lần lặp lại

- ❖ Đo lường sự cải thiện sau mỗi lần tối ưu hóa
- ❖ Nếu tối ưu hóa không cải thiện hiệu suất - hãy hoàn nguyên nó



18

Các kỹ thuật Code Tuning

- ❖ Ngừng kiểm tra khi bạn biết câu trả lời

```
if ( 5 < x ) and ( y < 10 ) then ...
```

```
?
```

```
negativeInputFound = False;  
for ( i = 0; i < iCount; i++ ) {  
    if ( input[ i ] < 0 ) {  
        negativeInputFound = True;  
    }  
}
```



19

Các kỹ thuật Code Tuning

- ❖ Sắp xếp các kiểm tra theo tần suất

```
Select char  
Case "+", "="  
    ProcessMathSymbol(char)  
Case "0" To "9"  
    ProcessDigit(char)  
Case ",", ".", "!", "?"  
    ProcessPunctuation(char)  
Case " "  
    ProcessSpace(char)  
Case "A" To "Z", "a" To "z"  
    ProcessAlpha(char)  
Case Else  
    ProcessError(char)  
End Select
```

```
Select char  
Case "A" To "Z", "a" To "z"  
    ProcessAlpha(char)  
Case " "  
    ProcessSpace(char)  
Case ",", ".", "!", "?"  
    ProcessPunctuation(char)  
Case "0" To "9"  
    ProcessDigit(char)  
Case "+", "="  
    ProcessMathSymbol(char)  
Case Else  
    ProcessError(char)  
End Select
```

20

Các kỹ thuật Code Tuning

❖ Unswitching loops

```
for ( i = 0; i < count; i++ ) {  
    if ( sumType == SUMTYPE_NET ) {  
        netSum = netSum + amount[ i ];  
    }  
    else { grossSum = grossSum + amount[ i ]; }  
}
```

?



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

21

21

Các kỹ thuật Code Tuning

❖ Giảm thiểu công việc bên trong các vòng lặp

```
for ( i = 0; i < rateCount; i++ ) {  
    netRate[i] = baseRate[i] * rates->discounts->factors->net;  
}
```

```
?  
for ( i = 0; i < rateCount; i++ ) {  
    netRate[i] = baseRate[i] * ?  
}
```



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

22

22

Các kỹ thuật Code Tuning

❖ Khởi tạo tại thời gian biên dịch

```
const double Log2 = 0.69314718055994529;
```



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

23

23

Các kỹ thuật Code Tuning

❖ Sử dụng Lazy Evaluation

```
public int getSize() {  
    if(size == null) {  
        size = the_series.size();  
    }  
    return size;  
}
```



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

24

24

Các kỹ thuật Code Tuning

```
var myClass = function() {  
    this.array_one = [1,2,3,4,5];  
    this.array_two = [1,2,3,4,5];  
    this.total = 0;  
}  
  
var my_instance = new myClass();  
  
for (var i=0; i < 4; i++) {  
    my_instance.total +=  
        (my_instance.array_one[i] +  
         my_instance.array_two[i]);  
}
```

Khi lặp qua dữ liệu, hãy giữ các tham chiếu bộ nhớ tuần tự

```
var myClass = function() {  
    this.array_one = [1,2,3,4,5];  
    this.array_two = [1,2,3,4,5];  
    this.total = 0;  
}  
  
var my_instance = new myClass();  
  
?
```

Nội dung

1. Phong cách lập trình
2. Tinh chỉnh / tối ưu mã (tuning / optimization)
- ⇒ 3. Tái cấu trúc mã (refactoring)
4. Debugging

Refactoring là gì?

Refactoring nghĩa là "
để cải thiện thiết kế và
chất lượng của mã
nguồn hiện tại mà
không thay đổi hành vi
bên ngoài của nó".

Martin Fowler



- ❖ Quy trình từng bước biến mã xấu thành mã tốt
 - Dựa trên "refactoring patterns" → các công thức nổi tiếng để cải thiện mã

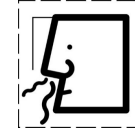
Code Refactoring

- Tái cấu trúc mã nguồn là gì?
 - Cải thiện thiết kế và chất lượng của mã nguồn hiện có mà không thay đổi hành vi của nó
 - Quy trình từng bước để biến mã xấu thành mã tốt (nếu có thể)
- Tại sao chúng ta cần tái cấu trúc?
 - Mã liên tục thay đổi và chất lượng của nó liên tục giảm (trừ khi được cấu trúc lại)
 - Các yêu cầu thường thay đổi và mã cần được thay đổi để tuân theo chúng

29

Khi nào thì Refactor?

- “Bad smells in the code” cho thấy cần phải cấu trúc lại
- Refactor:
 - Để thêm một chức năng mới dễ dàng hơn
 - Là một phần của quá trình sửa lỗi
 - Khi xem lại mã của người khác
 - Có món nợ kỹ thuật (hoặc bất kỳ mã có vấn đề nào)
 - Khi thực hiện phát triển dựa trên kiểm thử
- Unit tests đảm bảo rằng việc tái cấu trúc không thay đổi hành vi
 - Nếu không có kiểm thử đơn vị nào, hãy viết chúng



30

Refactoring: các nguyên lý chính

- Giữ mọi thứ đơn giản - Keep it simple (KISS principle)
- Tránh trùng lặp - Avoid duplication (DRY principle)
- Make it expressive (self-documenting, comments, etc.)
- Giảm mã tổng thể (KISS principle)
- Phân tách mối quan tâm - Separate concerns (decoupling)
- Mức độ trừu tượng phù hợp (làm việc thông qua các mức trừu tượng hóa)
- Quy tắc hướng đạo sinh (Boy scout rule)
 - Để lại mã của bạn tốt hơn sau bạn tìm thấy nó

31

Refactoring: quy trình điển hình

1. Hãy lưu lại mã bạn bắt đầu
 - Check-in hoặc sao lưu mã hiện tại
2. Chuẩn bị các tests để đảm bảo hành vi sau khi mã được cấu trúc lại
 - Unit tests / characterization tests
3. Thực hiện refactoring từng cái một
 - Thực hiện tái cấu trúc nhỏ
 - Đừng đánh giá thấp những thay đổi nhỏ
4. Chạy các tests và chúng nên pass / hoặc hoàn nguyên
5. Check-in (vào hệ thống kiểm soát mã nguồn)



32

Các mẹo Refactoring

- ❖ Hãy refactoring trên các phần nhỏ
- ❖ Từng lượt một (One at a time)
- ❖ Tạo danh sách checklist
- ❖ Tạo một "later" / TODO list
- ❖ Thường xuyên check-in / commit
- ❖ Thêm các tests cases
- ❖ Xét duyệt kết quả
 - Pair programming
- ❖ Sử dụng các công cụ (Visual Studio + add-ins / Eclipse + plugins / others)



 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

33

33

 ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



Code Refactoring

Live Demo

 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

34

34

Code Smells

- Code smells (code thối / mã xấu) == các cấu trúc nhất định trong mã dẫn đến khả năng tái cấu trúc
- Các loại code smells:
 - The bloaters
 - The obfuscators
 - Object-oriented abusers
 - Change preventers
 - Dispensables
 - The couplers



 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

35

35

Code Smells: The Bloaters

- Phương thức dài (Long method)
 - Các phương pháp nhỏ luôn tốt hơn (đặt tên dễ dàng, dễ hiểu, mã ít trùng lặp)
- Lớp lớn (Large class)
 - Quá nhiều các biến hoặc phương thức thể hiện
 - Vi phạm nguyên lý "Single Responsibility"
- Primitive obsession (lạm dụng primitives)
 - Sử dụng quá mức các giá trị nguyên thủy, thay vì trừu tượng hóa tốt hơn
 - Can be extracted in separate class with encapsulated validation



 VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

36

36

Code Smells: The Bloaters (2)

- Danh sách tham số dài - Long parameter list (**in / out / ref parameters**)
 - Có thể biểu thị phong cách thủ tục hơn là OO
 - Có thể là phương thức đang làm quá nhiều thứ
- Các khối dữ liệu (Data clumps)
 - Tập hợp dữ liệu luôn được sử dụng cùng nhau, nhưng không được tổ chức cùng nhau
 - E.g. trường thẻ tín dụng trong lớp Đơn hàng
- Bùng nổ tổ hợp
 - Ex. `ListCars()`, `ListByRegion()`, `ListByManufacturer()`, `ListByManufacturerAndRegion()`, etc.
 - Giải pháp có thể là Interpreter pattern (LINQ)



37

Code Smells: The Bloaters (3)

- Oddball solution
 - Một cách khác để giải quyết một vấn đề chung
 - Không sử dụng tính nhất quán
 - Giải pháp: Thay thế thuật toán hoặc sử dụng Adapter
- Class doesn't do much
 - Giải pháp: Hợp nhất với một lớp khác hoặc xóa bỏ
- Đòi hỏi setup / teardown code
 - Yêu cầu một số dòng mã trước khi sử dụng
 - Giải pháp: sử dụng đối tượng tham số, factory method, `IDisposable`



38

Code Smells: The Obfuscators

- Regions
 - Mục đích của mã không rõ ràng và cần nhận xét (smell)
 - Mã quá dài để hiểu (smell)
 - Giải pháp: partial class, a new class, organize code
- Comments
 - Nên dùng để nói **WHY**, not **WHAT** or **HOW**
 - Good comments: cung cấp thông tin bổ sung, liên kết đến vấn đề, giải thích thuật toán, giải thích lý do, đưa ra ngữ cảnh
 - Link: [Funny comments](#)



39

Code Smells: The Obfuscators (2)

- Tên kém / không phù hợp (Poor / improper names)
 - Phải phù hợp, mô tả và nhất quán
- Vertical separation
 - Bạn nên xác định các biến ngay trước khi sử dụng lần đầu tiên để tránh cuộn
 - Trong JS, các biến được định nghĩa khi bắt đầu hàm → sử dụng các hàm nhỏ
- Inconsistency
 - Thực hiện theo POLA (Nguyên lý ít suy giảm nhất)
 - Sự không nhất quán gây nhầm lẫn và mất tập trung
- Obscured intent
 - Mã phải càng diễn đạt càng tốt



40

Code Smells: OO Abusers

- Switch statement
 - Có thể được thay thế bằng tính đa hình
- Temporary field
 - Khi truyền dữ liệu giữa các phương thức
- Class depends on subclass
 - Các lớp không thể tách rời nhau (phụ thuộc vòng tròn)
 - Có thể phá vỡ nguyên lý thay thế Liskov
- Inappropriate static field
 - Strong coupling giữa static và callers
 - Những thứ tĩnh không thể thay thế hoặc sử dụng lại

41

Code Smells: Change Preventers

- Divergent change
 - Một lớp thường được thay đổi theo nhiều cách khác nhau / lý do khác nhau
 - Vi phạm SRP (single responsibility principle)
 - Giải pháp: extract class
- Shotgun surgery
 - Một thay đổi đòi hỏi nhiều thay đổi trong nhiều lớp
 - Khó tìm chúng, dễ bỏ sót một số
 - Giải pháp: di chuyển các phương thức, di chuyển các trường, tổ chức lại mã

42

Code Smells: Change Preventers (2)

- Conditional complexity
 - Độ phức tạp theo chu trình - Cyclomatic complexity (số lượng đường dẫn duy nhất mà mã có thể được đánh giá)
 - Các hiện tượng: deep nesting (arrow code) and buggy if-s
 - Các giải pháp: extract method, "Strategy" pattern, "State" pattern, "Decorator"
- Poorly written tests
 - Các test viết không tốt có thể ngăn cản sự thay đổi
 - Khớp nối chặt chẽ (Tight coupling)

43

Code Smells: Dispensables

- Lazy class
 - Các lớp không đủ để chứng minh sự tồn tại của chúng nên bị xóa
 - Mỗi lớp đều tốn chi phí để hiểu và duy trì
- Data class
 - Một số lớp chỉ có trường và thuộc tính
 - Thiếu sự công nhận? Lớp logic tách thành các lớp khác?
 - Giải pháp: chuyển logic liên quan vào lớp

44

Code Smells: Dispensables (2)

❖ Duplicated code

- Vi phạm nguyên lý DRY
- Kết quả của việc copy-paste code
- Giải pháp: extract method, extract class, pull-up method, **Template Method** pattern

❖ Dead code (mã không bao giờ sử dụng)

- Thường được phát hiện bởi các công cụ phân tích tĩnh

❖ Speculative generality

- "Một ngày nào đó chúng ta có thể cần cái này..."
- Nguyên tắc "YAGNI"

45

Code Smells: The Couplers

• Feature envy

- Phương thức có vẻ quan tâm hơn đến một lớp khác với lớp mà nó thực sự có trong
- Những điều cùng nhau thay đổi nên được đặt cùng nhau

• Inappropriate intimacy

- Những lớp biết quá nhiều về nhau
- Smells: thừa kế, các mối quan hệ hai chiều
- Giải pháp: move method / field, extract class, change bidirectional to unidirectional association, replace inheritance with delegation

46

Code Smells: The Couplers (2)

• The Law of Demeter (LoD)

- Một đối tượng nhất định phải giả định càng ít càng tốt về cấu trúc hoặc thuộc tính của bất kỳ thứ gì khác
- Bad e.g.: `customer.Wallet.RemoveMoney()`

• Indecent exposure

- Một số lớp hoặc thành viên là công khai nhưng thực tế không nên là như vậy
- Vi phạm tính năng đóng gói
- Có thể dẫn đến sự thân mật không phù hợp

47

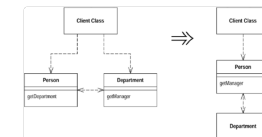
Code Smells: The Couplers (3)

• Message chains

- **Something.Another.SomeOther.Other.YetAnother**
- Khớp nối chặt chẽ giữa máy khách và cấu trúc của điều hướng

• Middle man

- Đôi khi việc ủy quyền đi quá xa
- Đôi khi chúng ta có thể xóa nó hoặc inline



• Tramp data

- Chuyển dữ liệu chỉ vì thứ khác cần nó
- Solutions: Remove middle-man data, extract class

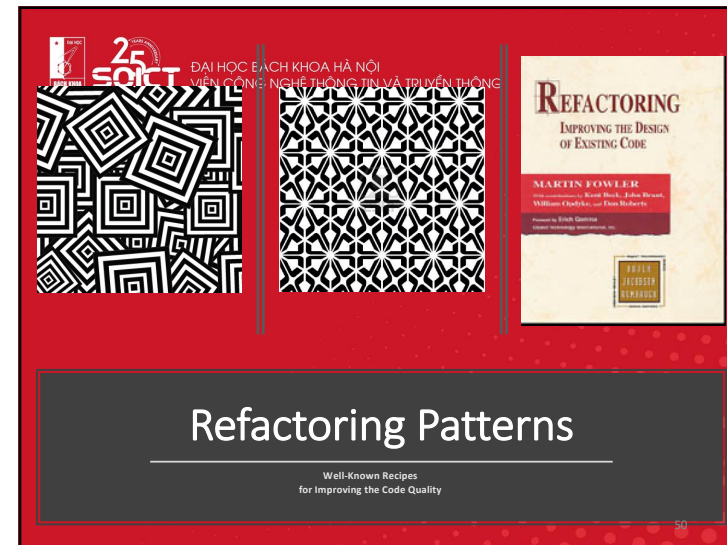


48

Code Smells: The Couplers (4)

- **Artificial coupling**
 - Những thứ không phụ thuộc vào nhau không nên được ghép nối một cách giả tạo
- **Hidden temporal coupling**
 - Các hoạt động được thực hiện liên tiếp không nên đoán
 - E.g. pizza class should not know the steps of making pizza -> Template Method pattern
- **Hidden dependencies**
 - Các lớp nên khai báo các phụ thuộc của chúng trong hàm khởi tạo
 - **new** is glue / Dependency Inversion principle

49



50

Refactoring Patterns

- Khi nào chúng ta nên thực hiện cấu trúc lại mã?
 - Bad smells trong mã cho thấy cần phải cấu trúc lại
- Các kiểm thử đơn vị đảm bảo rằng việc tái cấu trúc sẽ duy trì hành vi
- Refactoring patterns
 - Các đoạn mã lặp lại lớn → trích xuất mã trùng lặp trong các phương thức riêng biệt
 - Large methods → phân chia chúng một cách hợp lý
 - Thân vòng lớn hoặc lồng nhau sâu → extract method

51

Refactoring Patterns (2)

- Lớp hoặc phương thức có tính liên kết yếu → tách thành nhiều lớp / phương thức
- Thay đổi đơn lẻ thực hiện thay đổi trong một số lớp → các lớp có sự liên kết chặt chẽ với nhau → xem xét thiết kế lại
- Dữ liệu liên quan luôn được sử dụng cùng nhau nhưng không phải là một phần của một lớp đơn lẻ → nhóm chúng trong một lớp
- Một phương thức có quá nhiều tham số → tạo một lớp để nhóm các tham số lại với nhau
- Một phương thức gọi nhiều phương thức từ lớp khác hơn là từ lớp của chính nó → di chuyển nó

52

Raactoring Patterns (3)

- Hai lớp được kết hợp chặt chẽ với nhau → hợp nhất chúng hoặc thiết kế lại chúng để tách biệt trách nhiệm của chúng
- Các trường công khai không phải hằng số → đặt chúng ở chế độ riêng tư và xác định các thuộc tính truy cập
- Magic numbers trong mã → xem xét trích xuất các hằng số
- Lớp / phương thức / biến có tên không hợp lệ → đổi tên nó
- Điều kiện boolean phức tạp → chia nó thành một số biểu thức hoặc lệnh gọi phương thức

53

Raactoring Patterns (4)

- Biểu thức phức tạp → chia nó thành một vài phần đơn giản
- Một tập hợp các hằng được sử dụng làm kiểu liệt kê → chuyển đổi nó thành kiểu liệt kê
- Logic phương thức quá phức tạp → trích xuất một số phương thức đơn giản hơn hoặc thậm chí tạo một lớp mới
- Các lớp, phương thức, tham số, biến không sử dụng → loại bỏ chúng
- Dữ liệu lớn được truyền theo giá trị mà không có lý do chính đáng → chuyển nó theo tham chiếu

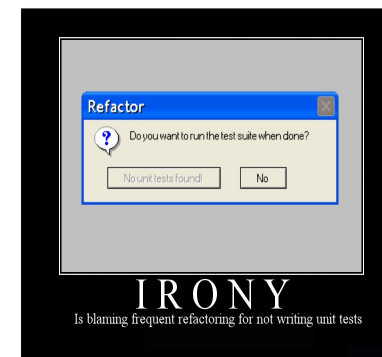
54

Raactoring Patterns (5)

- Một số lớp chia sẻ chức năng lặp lại → trích xuất lớp cơ sở và sử dụng lại mã chung
- Các lớp khác nhau cần được khởi tạo tùy thuộc vào cài đặt cấu hình → sử dụng factory
- Mã không được định dạng tốt → định dạng lại nó
- Quá nhiều lớp trong một không gian tên duy nhất → chia các lớp một cách hợp lý thành nhiều không gian tên hơn
- Không sử dụng các định nghĩa → loại bỏ chúng
- Các thông báo lỗi không mang tính mô tả → cải thiện chúng
- Không có lập trình phòng thủ → thêm nó

55

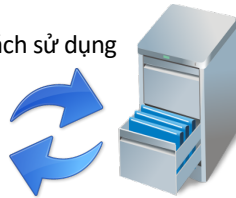
Refactoring Levels



56

Data-Level Refactoring

- ❖ Thay một số ma thuật bằng một hằng số được đặt tên
- ❖ Đổi tên một biến bằng tên nhiều thông tin hơn
- ❖ Thay thế một biểu thức bằng một phương thức
 - Để đơn giản hóa nó hoặc tránh trùng lặp mã
- ❖ Di chuyển một biểu thức inline
- ❖ Giới thiệu một biến trung gian
 - Giới thiệu biến giải thích
- ❖ Chuyển đổi một biến sử dụng nhiều lần thành nhiều biến sử dụng một lần
 - Tạo biến riêng biệt cho từng cách sử dụng



57

Data-Level Refactoring (2)

- Tạo một biến cục bộ cho các mục đích cục bộ thay vì một tham số
- Chuyển đổi một dữ liệu nguyên thủy thành một lớp
 - Hành vi bổ sung / logic xác thực (tiền)
- Chuyển đổi một tập mã kiểu (hằng số) thành enum
- Chuyển đổi một tập mã kiểu thành một lớp với các lớp con có hành vi khác nhau
- Thay đổi một mảng thành một đối tượng
 - Khi bạn sử dụng một mảng có các kiểu khác nhau trong đó
- Đóng gói một tập hợp



58

Statement-Level Refactoring

- Phân rã một biểu thức boolean
- Di chuyển một biểu thức boolean phức tạp thành một hàm boolean được đặt tên tốt
- Sử dụng break hoặc return thay vì biến điều khiển vòng lặp
- Trả lại ngay khi bạn biết câu trả lời thay vì chỉ định giá trị trả về
- Hợp nhất mã trùng lặp trong điều kiện
- Thay thế các điều kiện bằng đa hình
- Sử dụng mẫu thiết kế đối tượng null thay vì kiểm tra null



59

Method-Level Refactoring

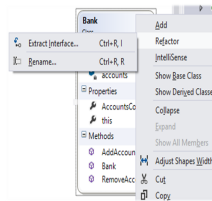
- ❖ Trích xuất Phương thức / nội tuyến phương thức
- ❖ Đổi tên một phương thức
- ❖ Chuyển một phương thức dài thành một lớp
- ❖ Thêm / bớt tham số
- ❖ Kết hợp các phương thức tương tự bằng cách tham số hóa chúng
- ❖ Thay thế một thuật toán phức tạp bằng một thuật toán đơn giản hơn
- ❖ Các phương thức riêng biệt có hành vi phụ thuộc vào các tham số được truyền vào (tạo các phương thức mới)
- ❖ Chuyển toàn bộ đối tượng thay vì các trường cụ thể
- ❖ Đóng gói downcast / return các kiểu giao diện



60

Class-Level Refactoring

- ❖ Thay đổi cấu trúc thành lớp và ngược lại
- ❖ Kéo các thành viên lên / đẩy các thành viên xuống hệ thống phân cấp
- ❖ Trích xuất mã chuyên biệt thành một lớp con
- ❖ Kết hợp mã tương tự vào một lớp cha
- ❖ Thu gọn hệ thống phân cấp
- ❖ Thay thế kế thừa bằng ủy quyền
- ❖ Thay thế ủy quyền bằng kế thừa



61

Class Interface Refactorings

- Trích xuất (các) giao diện / giữ phân tách giao diện
- Di chuyển một phương thức sang một lớp khác
- Tách một lớp / hợp nhất các lớp / xóa một lớp
- Ẩn lớp ủy nhiệm
 - A gọi B và C khi A nên gọi B và B gọi C
- Loại bỏ người đàn ông ở giữa
- Giới thiệu (sử dụng) một lớp mở rộng
 - Khi bạn không có quyền truy cập vào lớp ban đầu
 - Hoặc sử dụng mẫu Decorator

62

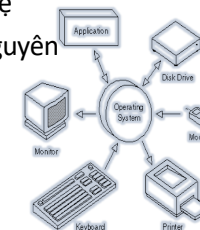
Class Interface Refactoring (2)

- ❖ Đóng gói một biến thành viên bị bọc lộ
 - Luôn sử dụng thuộc tính
 - Xác định quyền truy cập thích hợp vào getters và setters
 - Xóa setters thành dữ liệu chỉ đọc
- ❖ Ẩn dữ liệu và quy trình không nhằm mục đích sử dụng bên ngoài lớp / cấu trúc phân cấp
 - private -> protected -> internal -> public
- ❖ Sử dụng chiến lược để tránh phân cấp lớp lớn
- ❖ Áp dụng các mẫu thiết kế khác để giải quyết các vấn đề phổ biến về phân cấp lớp và lớp (Facade, Adapter, v.v.)

63

System-Level Refactoring

- ❖ Di chuyển lớp (tập hợp các lớp) sang không gian tên / hợp ngữ khác
- ❖ Cung cấp một phương thức gốc thay vì một hàm tạo đơn giản / sử dụng API thông thạo
- ❖ Thay thế mã lỗi bằng các ngoại lệ
- ❖ Trích xuất chuỗi thành tệp tài nguyên
- ❖ Sử dụng dependency injection
- ❖ Áp dụng các mẫu kiến trúc



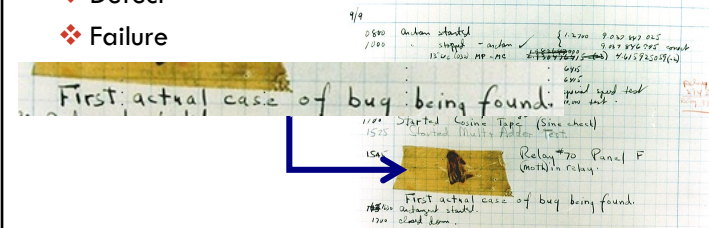
64

Nội dung

1. Phong cách lập trình
2. Tinh chỉnh / tối ưu mã (tuning / optimization)
3. Tái cấu trúc mã (refactoring)
- ➔ 4. Debugging

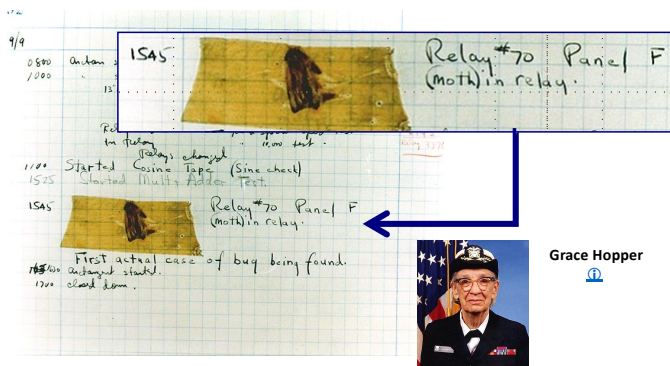
4.1. Tổng quan

- ❖ Error
- ❖ Bug
- ❖ Fault
- ❖ Defect
- ❖ Failure



Khi nào “bug”

- ❖ Alan Perlis



4.2. Phòng thủ theo chiều sâu

- ❖ Không thể xảy ra lỗi
 - Java làm cho lỗi ghi đè bộ nhớ không thể xảy ra
- ❖ Không giới thiệu các khiếm khuyết
 - Tính đúng đắn: làm mọi thứ đúng ngay lần đầu tiên
- ❖ Hiện thị lỗi ngay lập tức
 - Khả năng hiển thị lỗi cục bộ: tốt nhất là không thành công ngay lập tức
 - Ví dụ: assertions
- ❖ Phương án cuối cùng là gỡ lỗi
 - Cần thiết khi thất bại (ảnh hưởng) khác xa với nguyên nhân (khuyết tật)
 - Phương pháp khoa học: Thiết kế các thí nghiệm để thu thập thông tin về khuyết tật
 - Khả dễ dàng trong một chương trình có mô-đun tốt, ẩn biểu diễn, thông số kỹ thuật, kiểm tra đơn vị, v.v.
 - Khó hơn và khó hơn nhiều với một thiết kế kém, ví dụ: với phor sáng trần lan

4.2.1. Phòng thủ đầu tiên: Không thể theo thiết kế

- ❖ Bằng ngôn ngữ
 - Java làm cho lỗi ghi đè bộ nhớ không thể xảy ra
- ❖ Trong các giao thức / thư viện / mô-đun
 - TCP / IP đảm bảo rằng dữ liệu không được sắp xếp lại
 - BigInteger đảm bảo rằng không có tràn
- ❖ Trong các quy ước tự áp đặt
 - Cấm đệ quy ngăn chặn đệ quy vô hạn / không đủ ngăn xếp - mặc dù nó có thể đẩy vấn đề sang chỗ khác
 - Cấu trúc dữ liệu bất biến đảm bảo bình đẳng về hành vi
 - Thận trọng: Bạn phải duy trì kỷ luật

4.2.2. Phòng thủ thứ hai: Tính đúng đắn

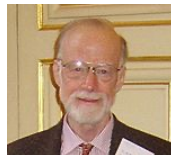
- ❖ Làm mọi thứ đúng ngay lần đầu tiên
 - Hãy suy nghĩ trước khi viết mã. Đừng viết mã trước khi bạn nghĩ!
 - Nếu bạn đang tạo ra nhiều khiếm khuyết để tìm, thì bạn cũng đang tạo ra những khiếm khuyết khó tìm - đừng sử dụng trình biên dịch như một cái nạng
- ❖ Đặc biệt đúng, khi gỡ lỗi sẽ khó
 - Đồng thời, mỗi trường thời gian thực, không có quyền truy cập vào môi trường khách hàng, v.v.
- ❖ Đơn giản là chìa khóa
 - Môđun hoá (Modularity)
 - Chia chương trình thành nhiều phần dễ hiểu
 - Sử dụng các kiểu dữ liệu trừu tượng với giao diện được xác định rõ
 - Sử dụng chương trình phòng thủ; tránh tiếp xúc đại diện
 - Đặc tả (Specification)
 - Viết thông số kỹ thuật cho tất cả các mô-đun để có một hợp đồng rõ ràng, được xác định rõ ràng giữa mỗi mô-đun và các khách hàng của nó

Cố gắng vì sự đơn giản

"There are two ways of constructing a software design:

- ❖ One way is to make it so simple that there are obviously no deficiencies, and
- ❖ the other way is to make it so complicated that there are no obvious deficiencies.

The first method is far more difficult."



Sir Anthony Hoare



Brian Kernighan



"Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."

4.2.3. Phòng thủ thứ ba: Khả năng hiển thị ngay lập tức

- ☐ Nếu chúng ta không thể ngăn lỗi, chúng ta có thể cố gắng bản địa hóa chúng thành một phần nhỏ của chương trình
 - ☐ Assertions: phát hiện lỗi sớm, trước khi chúng lây nhiễm và có thể được che giấu bằng cách tính toán thêm
 - ☐ Kiểm thử đơn vị: khi bạn kiểm tra riêng một mô-đun, bạn có thể tự tin rằng bất kỳ lỗi nào bạn tìm thấy là do lỗi trong đơn vị đó (trừ khi nó nằm trong trình điều khiển kiểm tra)
 - ☐ Kiểm thử hồi quy: chạy kiểm tra thường xuyên nhất có thể khi thay đổi mã. Nếu không thành công, rất có thể có lỗi trong mã bạn vừa thay đổi
- ☐ Khi bản địa hóa thành một phương thức hoặc mô-đun nhỏ, các khiếm khuyết thường có thể được tìm thấy đơn giản bằng cách nghiên cứu văn bản chương trình

Lợi ích của việc hiển thị ngay lập tức

- ❖ Khó khăn chính của việc gỡ lỗi là tìm ra lỗi: đoạn mã chịu trách nhiệm cho một vấn đề được quan sát
 - Một phương thức có thể trả về một kết quả sai, nhưng bản thân nó không có lỗi, nếu có sự thay đổi trước đó về đại diện
- ❖ Sự cố được quan sát càng sớm thì càng dễ sửa chữa
 - Thường xuyên kiểm tra bất biến đại diện giúp
- ❖ Cách tiếp cận chung: không nhanh
 - Kiểm tra các bất biến, đừng chỉ giả định chúng
 - Đừng (thường) cố gắng khôi phục lỗi - nó có thể chỉ che giấu chúng

73

Đừng che giấu lỗi

```
// k is guaranteed to be present in a
int i = 0;
while (true) {
    if (a[i]==k) break;
    i++;
}
```

- ❖ Đoạn mã này tìm kiếm một mảng a cho một giá trị k.
 - Giá trị được đảm bảo nằm trong mảng
 - Điều gì sẽ xảy ra nếu bảo đảm đó bị phá vỡ (do khiếm khuyết)?
- ❖ Cạm bẫy: làm cho mã “mạnh mẽ hơn” bằng cách không thất bại

74

Đừng che giấu lỗi

```
// k is guaranteed to be present in a
int i = 0;
while (i < a.length) {
    if (a[i] == k) break;
    i++;
}
```

- ❑ Bây giờ ít nhất vòng lặp sẽ luôn kết thúc
 - ❑ Nhưng không còn được đảm bảo rằng $a[i] == k$
 - ❑ Nếu mã khác dựa vào điều này, thì các vấn đề phát sinh sau đó
 - ❑ Điều này làm cho khó thấy mối liên hệ giữa sai sót và hỏng hóc

75

Đừng che giấu lỗi

```
// k is guaranteed to be present in a
int i = 0;
while (i < a.length) {
    if (a[i] == k) break;
    i++;
}
assert (i == a.length) : "key not found";
```

- ❖ Các khẳng định cho phép chúng ta ghi lại và kiểm tra các bất biến
 - Hủy bỏ / gỡ lỗi chương trình ngay khi phát hiện sự cố: biến một lỗi thành một thất bại
- ❖ Nhưng xác nhận không được kiểm tra cho đến khi chúng ta sử dụng dữ liệu, có thể rất lâu sau lỗi ban đầu
 - "Tại sao không phải là khóa trong mảng?"

76

Kiểm tra Production Code

- ❖ Bạn có nên đưa các xác nhận và kiểm tra vào mã sản xuất không?
 - Có: dừng chương trình nếu kiểm tra không thành công - không muốn để mất cơ hội chương trình sẽ làm sai
 - Không: có thể cần chương trình để tiếp tục, có thể lỗi không gây ra hậu quả xấu như vậy (lỗi có thể chấp nhận được)
 - Câu trả lời đúng phụ thuộc vào ngữ cảnh!

Ariane 5 - chương trình bị tạm dừng vì tràn giá trị chưa sử dụng, trường hợp ngoại lệ được ném ra nhưng không được xử lý cho đến cấp cao nhất, sự cố tên lửa... [mặc dù câu chuyện đầy đủ phức tạp hơn]



4.2.3. Gỡ lỗi: Phương sách cuối cùng

- ❖ Khiếm khuyết xảy ra - con người không hoàn hảo
 - Mức trung bình trong ngành: 10 lỗi trên 1000 dòng mã ("kloc")
- ❖ Các khiếm khuyết không thể bản địa hóa ngay lập tức xảy ra
 - Tìm thấy trong quá trình kiểm tra tích hợp
 - Hoặc do người dùng báo cáo
- ❖ Chi phí tìm và sửa lỗi thường tăng lên theo thứ tự độ lớn cho mỗi giai đoạn vòng đời mà nó đi qua
- ❖ bước 1 - Làm rõ triệu chứng (đơn giản hóa đầu vào), tạo kiểm tra
- ❖ bước 2 - Tìm và hiểu nguyên nhân, tạo thử nghiệm tốt hơn
- ❖ bước 3 - Sửa chữa
- ❖ bước 4 - Chạy lại tất cả các bài kiểm tra

4.3. Debugging

- ❖ Bước 1 - tìm một trường hợp thử nghiệm nhỏ, có thể lặp lại tạo ra lỗi (có thể tốn nhiều công sức, nhưng giúp làm rõ lỗi và cũng cung cấp cho bạn một cái gì đó để hồi quy)
 - Đừng chuyển sang bước tiếp theo cho đến khi bạn có một test có thể lặp lại
- ❖ Bước 2 - thu hẹp vị trí và nguyên nhân lân cận
 - Nghiên cứu dữ liệu / giả thuyết / thử nghiệm / lặp lại
 - Có thể thay đổi mã để có thêm thông tin
 - Đừng chuyển sang bước tiếp theo cho đến khi bạn hiểu nguyên nhân
- ❖ Bước 3 - sửa lỗi
 - Đó có phải là lỗi đánh máy đơn giản, hay lỗi thiết kế? Nó có xảy ra ở nơi khác không?
- ❖ Bước 4 - thêm trường hợp thử nghiệm vào bộ hồi quy
 - Lỗi này có được khắc phục không? Có bất kỳ lỗi mới nào khác được giới thiệu không?

Gỡ lỗi và phương pháp khoa học

- ❖ Gỡ lỗi phải có hệ thống
 - Hãy cẩn thận quyết định xem phải làm gì - thất bại có thể là một trường hợp thất bại trong sử thi
 - Ghi lại mọi thứ bạn làm
 - Đừng để bị hút vào những con đường không có kết quả
- ❖ Hình thành giả thuyết
- ❖ Thiết kế một thử nghiệm
- ❖ Thực hiện thử nghiệm
- ❖ Điều chỉnh giả thuyết của bạn và tiếp tục



Ví dụ về giảm kích thước đầu vào

```
// returns true iff sub is a substring of full
// (i.e. iff there exists A,B s.t. full=A+sub+B)
boolean contains(String full, String sub);
```

- ❖ Báo cáo lỗi người dùng
 - không thể tìm thấy chuỗi "very happy" trong:
"Fáilte, you are very welcome! Hi Seán! I am very very happy to see you all."
- ❖ Ít phản hồi lý tưởng
 - Xem các ký tự có dấu, về việc không nghĩ đến unicode và tìm hiểu các văn bản Java của bạn để xem cách xử lý
 - Cố gắng theo dõi việc thực hiện ví dụ này
- ❖ Phản hồi tốt hơn: đơn giản hóa / làm rõ các triệu chứng

81

Giảm kích thước đầu vào tuyệt đối

- ❖ Tìm một trường hợp thử nghiệm đơn giản bằng cách chia để trị
- ❖ Pare test down – can't find "very happy" within
 - "Fáilte, you are very welcome! Hi Seán! I am very very happy to see you all."
 - "I am very very happy to see you all."
 - "very very happy"
- ❖ Can find "very happy" within
 - "very happy"
- ❖ Can't find "ab" within "aab"

82

Giảm kích thước đầu vào tương đối

- ❖ Đôi khi sẽ rất hữu ích nếu bạn tìm thấy hai trường hợp thử nghiệm gần như giống hệt nhau, trong đó một trường hợp đưa ra câu trả lời chính xác và trường hợp còn lại thì không
 - Can't find "very happy" within
 - "I am very very happy to see you all."
 - Can find "very happy" within
 - "I am very happy to see you all."

83

Chiến lược chung: đơn giản hóa

- ❖ Nói chung: tìm đầu vào đơn giản nhất sẽ dẫn đến thất bại
 - Thường không phải là đầu vào tiết lộ sự tồn tại của khiếm khuyết
- ❖ Bắt đầu với dữ liệu tiết lộ lỗi
 - Tiếp tục phân tích nó (tìm kiếm nhị phân "bởi bạn" có thể giúp ích)
 - Thường dẫn trực tiếp đến sự hiểu biết về nguyên nhân
- ❖ Khi không xử lý các cuộc gọi phương thức đơn giản
 - "Đầu vào kiểm tra" là tập hợp các bước gây ra lỗi một cách đáng tin cậy
 - Cùng một ý tưởng cơ bản

84

Bản địa hóa một khiếm khuyết

❑ Tận dụng tính mô-đun

- ❑ Bắt đầu với mọi thứ, lấy đi từng phần cho đến khi thất bại
- ❑ Bắt đầu mà không có gì, thêm các mảnh trở lại cho đến khi thất bại xuất hiện

❑ Tận dụng lý luận mô-đun

- ❑ Theo dõi chương trình, xem kết quả trung gian

❑ Tìm kiếm nhị phân tăng tốc quá trình

- ❑ Lỗi xảy ra ở đâu đó giữa câu lệnh đầu tiên và câu lệnh cuối cùng
- ❑ Thực hiện tìm kiếm nhị phân trên tập hợp các câu lệnh có thứ tự đó

Tìm kiếm nhị phân trên mã lỗi

```
public class MotionDetector {  
    private boolean first = true;  
    private Matrix prev = new Matrix();
```

```
    public Point apply(Matrix current) {  
        if (first) {  
            prev = current;  
        }  
        Matrix motion = new Matrix();  
        getDifference(prev, current, motion);  
        applyThreshold(motion, motion, 10);  
        labelImage(motion, motion);  
        Hist hist = getHistogram(motion);  
        int top = hist.getMostFrequent();  
        applyThreshold(motion, motion, top, top);  
        Point result = getCentroid(motion);  
        prev.copy(current);  
        return result;  
    }  
}
```

no problem yet

Check
intermediate result
at half-way point

problem exists

Tìm kiếm nhị phân trên mã lỗi

```
public class MotionDetector {  
    private boolean first = true;  
    private Matrix prev = new Matrix();
```

```
    public Point apply(Matrix current) {  
        if (first) {  
            prev = current;  
        }  
        Matrix motion = new Matrix();  
        getDifference(prev, current, motion);  
        applyThreshold(motion, motion, 10);  
        labelImage(motion, motion);  
        Hist hist = getHistogram(motion);  
        int top = hist.getMostFrequent();  
        applyThreshold(motion, motion, top, top);  
        Point result = getCentroid(motion);  
        prev.copy(current);  
        return result;  
    }  
}
```

no problem yet

Check
intermediate result
at half-way point

problem exists

Quickly home in
on defect in $O(\log n)$ time
by repeated subdivision

Phát hiện lỗi trong thế giới thực

❖ Hệ thống thực

- Lớn và phức tạp (duh!)
- Bộ sưu tập các mô-đun, được viết bởi nhiều người
- Đầu vào phức tạp
- Nhiều tương tác bên ngoài
- Không xác định

❖ Sao chép có thể là một vấn đề

- Không thường xuyên

❖ Thiết bị đo lường loại bỏ lỗi

- Khiếm khuyết các rào cản trừu tượng chéo
- Độ trễ thời gian lớn từ corruption (defect) đến phát hiện (failure)

Heisenbugs

- ❖ Chương trình tuần tự, xác định - lỗi có thể lặp lại
- ❖ Nhưng thế giới thực không tốt đẹp như vậy...
 - Thay đổi đầu vào / môi trường liên tục
 - Thời gian phụ thuộc
 - Đồng thời và song song
- ❖ Thất bại xảy ra ngẫu nhiên
- ❖ Khó tái tạo
 - Sử dụng trình gỡ lỗi hoặc xác nhận → lỗi sẽ biến mất
 - Chỉ xảy ra khi chịu tải nặng
 - Chỉ thỉnh thoảng xảy ra một lần

89

Ghi nhật ký sự kiện

- ❖ Xây dựng nhật ký sự kiện (bộ đệm tròn) và ghi các sự kiện trong quá trình thực thi chương trình khi chương trình chạy ở tốc độ
- ❖ Khi phát hiện lỗi, hãy dừng chương trình và kiểm tra nhật ký để giúp bạn xây dựng lại quá khứ
- ❖ Nhật ký có thể là tất cả những gì bạn biết về môi trường của khách hàng - giúp bạn tái tạo lỗi

90

Thủ thuật cho lỗi khó

- ❖ Xây dựng lại hệ thống từ đầu hoặc khởi động lại / khởi động lại
 - Tìm lỗi trong hệ thống xây dựng hoặc cấu trúc dữ liệu liên tục của bạn
- ❖ Giải thích vấn đề cho một người bạn
- ❖ Đảm bảo rằng đó là một lỗi - chương trình có thể đang hoạt động chính xác mà bạn không nhận ra!
- ❖ Giảm thiểu đầu vào được yêu cầu để thực hiện lỗi (có lỗi)
- ❖ Thêm séc vào chương trình
 - Giảm thiểu khoảng cách giữa lỗi và phát hiện / lỗi
 - Sử dụng tìm kiếm nhị phân để thu hẹp các vị trí có thể
- ❖ Sử dụng nhật ký để ghi lại các sự kiện trong lịch sử

91

Lỗi ở đâu?

- ☐ Nếu lỗi không ở nơi bạn nghĩ, hãy tự hỏi mình xem nó không thể ở đâu; giải thích vì sao
- ☐ Trước tiên, hãy tìm những sai lầm ngớ ngẩn, ví dụ:
 - ☒ Reversed order of arguments: `Collections.copy(src, dest)`
 - ☒ Spelling of identifiers: `int hashCode()`
 - `@Override` can help catch method name typos
 - ☒ Same object vs. equal: `a == b` versus `a.equals(b)`
 - ☒ Failure to reinitialize a variable
 - ☒ Deep vs. shallow copy
- ☐ Đảm bảo rằng bạn có mã nguồn chính xác
 - ☐ Biên dịch lại mọi thứ

92

Khi mọi việc trở nên khó khăn

- ❖ Xem xét lại các giả định
 - Ví dụ: hệ điều hành có thay đổi không? Có chỗ trên ổ cứng không?
 - Gỡ lỗi mã, không phải nhận xét - đảm bảo nhận xét và thông số kỹ thuật mô tả mã
- ❖ Bắt đầu ghi lại hệ thống của bạn
 - Cung cấp một góc mới và làm nổi bật khu vực nhầm lẫn
- ❖ Được trợ giúp
 - Tất cả chúng ta đều phát triển những điểm mù
 - Giải thích vấn đề thường giúp
- ❖ Bỏ đi
 - Độ trễ giao dịch để đạt hiệu quả - đi ngủ!
 - Một lý do tốt để bắt đầu sớm

93

Các khái niệm chính được xem xét

- ❖ Kiểm tra và gỡ lỗi khác nhau
 - Kiểm tra cho thấy sự tồn tại của các lỗi
 - Gỡ lỗi xác định vị trí chính xác của các lỗi
- ❖ Mục tiêu là làm cho chương trình phù hợp
- ❖ Gỡ lỗi phải là một quá trình có hệ thống
 - Sử dụng phương pháp khoa học
- ❖ Hiểu nguồn gốc của khuyết tật
 - Để tìm những cái tương tự và ngăn chặn chúng trong tương lai

94

Công cụ hỗ trợ: Trình cài thêm Eclipse

- ❖ Checkstyle: Giúp lập trình viên viết mã Java tuân theo tiêu chuẩn viết mã
- ❖ FindBugs: Sử dụng phân tích tĩnh để tìm lỗi trong mã Java
 - Standalone Swing application
 - Eclipse plug-in
 - Integrated into the build process (Ant or Maven)

95

Các tính năng của Findbugs

- ❖ Không quan tâm đến các tiêu chuẩn định dạng hoặc mã hóa
- ❖ Phát hiện các lỗi tiềm ẩn và các vấn đề về hiệu suất
 - Có thể phát hiện nhiều loại lỗi phổ biến, khó tìm
 - Sử dụng “bug patterns”

NullPointerException

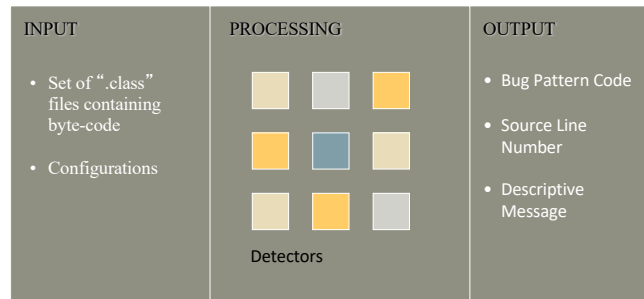
```
Address address = client.getAddress();
if ((address != null) || (address.getPostCode() != null)) {
    ...
}
```

Uninitialized field

```
public class ShoppingCart {
    private List items;
    public addItem(Item item) {
        items.add(item);
    }
}
```

96

Findbug hoạt động như thế nào?



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

97

97

Findbugs có thể làm gì?

- ❖ FindBugs đi kèm với hơn 200 quy tắc được chia thành các loại khác nhau:
 - Tính đúng đắn
 - Ví dụ. vòng lặp đệ quy vô hạn, đọc một trường không bao giờ được ghi
 - Thực hành không tốt
 - Ví dụ. mã giảm ngoại lệ hoặc không đóng được tệp
 - Hiệu suất
 - Độ đúng đa luồng
 - Tinh ranh
 - Ví dụ. biến cục bộ không sử dụng hoặc phôi không được kiểm tra



VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

98

98



99

 **SOICT** 25
ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Bài 10: Các khái niệm trong thiết kế

1

Nội dung

1. Cách thức thiết kế
2. Móc nối linh hoạt (low coupling)
3. Tính kết dính cao (high cohesion)

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

2

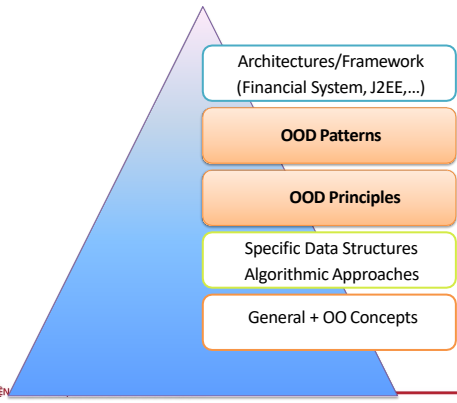
Nội dung

1. Cách thức thiết kế
2. Móc nối linh hoạt (low coupling)
3. Tính kết dính cao (high cohesion)

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

3

Các mức thiết kế



Architectures/Framework
(Financial System, J2EE,...)

OOD Patterns

OOD Principles

Specific Data Structures
Algorithmic Approaches

General + OO Concepts

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

4

Quá trình thiết kế

❖ Định nghĩa:

- Thiết kế là quá trình tìm và mô tả giải pháp
 - Thực thi các yêu cầu chức năng
 - Đảm bảo các yêu cầu phi chức năng
 - Bao gồm cả yêu cầu về ngân sách
 - Tuân thủ các nguyên lý đảm bảo chất lượng thiết kế

Thiết kế là phải ra các quyết định

❖ Người thiết kế phải đối mặt với rất nhiều vấn đề

- Mỗi vấn đề sẽ có một vài giải pháp
 - Các lựa chọn thiết kế
- Người thiết kế phải ra quyết định, chọn giải pháp cho từng vấn đề
 - Có nhiều lựa chọn
 - Giải pháp vấn đề này ảnh hưởng tới giải pháp của vấn đề khác

Ra quyết định

❖ Khi ra quyết định, người kiến trúc sư phần mềm phải sử dụng

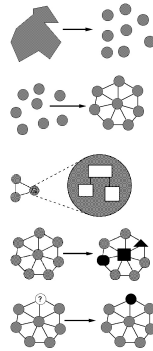
- Các yêu cầu phần mềm
- Các thiết kế đến hiện tại
- Các công nghệ sẵn có
- Các nguyên lý và hướng dẫn thiết kế phần mềm
- Kinh nghiệm trong quá khứ

Modules

- ❖ Phần mềm đều được thiết kế theo hướng module hóa
- ❖ Module là khái niệm tổng quát. Nó có thể là một phương thức, lớp, package hoặc là một đơn vị thiết kế bất kỳ nào
- ❖ Thiết kế module hóa tập trung vào việc định nghĩa các module, vào các đặc tả của chúng, cách chúng liên kết với nhau, không tập trung vào việc thực thi các module

Các tính chất cần đảm bảo khi thiết kế module hóa

- ❖ Tính tách rời được: hệ thống có thể được phân chia thành các module để giảm độ phức tạp, cho phép nhiều người cùng đồng thời làm việc, mỗi người phụ trách một module
- ❖ Tính tổ hợp được: Sau khi chia module, yêu cầu là có thể tích hợp các module lại thành một hệ thống
- ❖ Tính hiểu được: Mỗi module có thể được phát triển, kiểm tra, đánh giá một cách độc lập
- ❖ Tính liên tục theo thời gian: một thay đổi nhỏ trong yêu cầu chỉ ảnh hưởng đến một số lượng nhỏ các module
- ❖ Tính tách biệt: Lỗi trong một module phải bị cô lập theo cách thức tốt nhất có thể, không lan sang module khác



9

Thiết kế Top-down và bottom-up

❖ Thiết kế Top-down

- Trước tiên thiết kế cấu trúc hệ thống ở mức cao
- Sau đó đi chi tiết dần xuống các mức dưới
- Đi đến các quyết định chi tiết như
 - CTDL, thuật toán như thế nào

10

Thiết kế Top-down và bottom-up

❖ Thiết kế Bottom-up

- Xác định các thành phần mức thấp trước
- Sau đó tổ hợp lại dần dần thành các thành phần mức cao

❖ Nên kết hợp thiết kế top-down và bottom-up:

- Thiết kế Top-down giúp hệ thống có cấu trúc rõ ràng, tối ưu
- Thiết kế Bottom-up giúp tạo ra các thành phần có tính tái sử dụng cao

11

Các vấn đề cần thiết kế

- Thiết kế kiến trúc:
 - Chia thành các phần tử, các hệ thống con
 - Cách chúng kết nối với nhau
 - Cách chúng tương tác với nhau
 - Giao diện của chúng
- Thiết kế lớp
- Thiết kế giao diện
- Thiết kế thuật toán
- Thiết kế giao thức

12

Thiết kế tốt

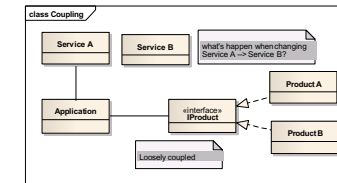
- ❖ Thiết kế tốt phải giúp giảm chi phí phát triển, giúp tăng lợi nhuận. Phải đảm bảo tuân thủ các yêu cầu phần mềm đặt ra, tăng tính tái sử dụng, để bảo trì, tăng hiệu năng, ...
- ❖ Để đánh giá thiết kế: có 2 nguyên lý cơ bản và hữu hiệu nhất về cohesion và coupling

13

Cohesion và Coupling

□ Tính móc nối/phụ thuộc - Coupling

Coupling/Dependency là mức độ 1 module phụ thuộc vào các module khác.



□ Tính kết dính – Cohesion

Cohesion là mức độ liên quan giữa các phần tử trong một module.

14

Các tính chất cần đạt được cho các modules

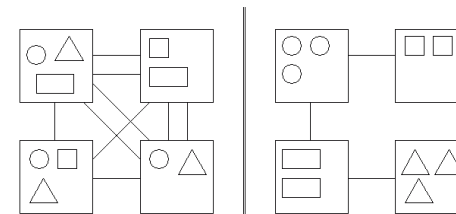
- ❖ Các modules nên độc lập với nhau
 - Tránh một module làm nhiều tác vụ
- ===> high COHESION (tính kết dính cao)
 - Một module nên tương tác ít nhất có thể với các module khác
 - Giao diện đơn giản
- ===> low COUPLING (tính phụ thuộc thấp/tính móc nối linh hoạt)

[Steven, Myers, Constantine]

15

Cohesion và Coupling

- ❖ Thiết kế tốt nhất là phải có tính kết dính cao (high cohesion hoặc strong cohesion) trong một module và móc nối linh hoạt lỏng lẻo (low coupling hoặc weak coupling) giữa các module.



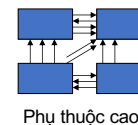
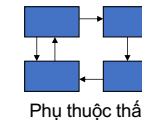
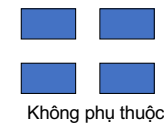
16

Nội dung

1. Cách thức thiết kế
2. **Mức nối linh hoạt (low coupling)**
3. Tính kết dính cao (high cohesion)

17

Coupling: Mức độ phụ thuộc giữa các phần tử



High coupling khiến việc sửa đổi các thành phần trong hệ thống rất khó khăn. Sửa một thành phần sẽ ảnh hưởng tới các thành phần khác kết nối tới nó

18

18

Đánh giá mức độ Coupling

- ❖ Degree
 - Degree: số lượng kết nối giữa 1 module với các module khác. Càng nhỏ càng tốt
- ❖ Ease
 - Ease: Mức độ dễ dàng khi kết nối 1 module với các module khác. Càng dễ hiểu, dễ làm càng tốt
- ❖ Flexibility
 - Flexibility: các module có thể dễ dàng được thay thế hay không

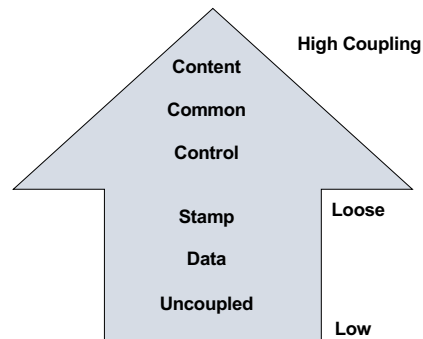
19

Tác hại của tight coupling

- ❖ Thay đổi một module, dẫn đến các thay đổi lan truyền ở các module khác
- ❖ Kết nối các module lại với nhau rất khó khăn
- ❖ Khó tái sử dụng, test các module

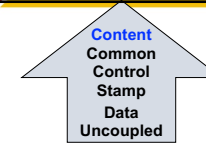
20

Các loại/mức độ Coupling



21

2.1. Content coupling (Móc nối nội dung)



❖ Định nghĩa: Một component tham chiếu đến nội dung của component khác

❖ Ví dụ:

- Component sửa dữ liệu của component khác
- Component chỉnh sửa code của component khác (NNLT bậc thấp)

22

Content coupling:

- ❖ Để tránh 1 component sửa dữ liệu nội bộ trong component khác, trong OOP:
 - Cần thực hiện nguyên lý đóng gói và che giấu dữ liệu, đảm bảo dữ liệu là private, cung cấp getter, setter hợp lý
- ❖ Lợi ích: nếu f thay đổi biến cục bộ của g, khi sửa f ta phải sửa g và ngược lại

23

Ví dụ 1

❖ Hàm bạn trong C++

```
class A {
private:
    int a;
public:
    A() { a = 0; }
    friend class B; // Friend Class
};

class B {
private:
    int b;
public:
    void showA(A& x)
    {
        // Since B is friend of A, it can access
        // private members of A
        std::cout << "A::a=" << x.a;
    }
};
```

24

Ví dụ 2

- ❖ "getters" và "setters" có thể vẫn chưa đảm bảo ngăn ngừa content coupling

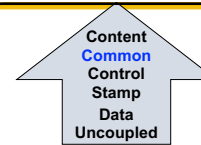
```
public int sumValues(Calculator c){ // tight coupling
    int result = c.getFirstNumber() + c.getSecondNumber();
    c.setResult(result);
    return c.getResult();
}

public int sumValues(Calculator c){ // loose coupling
    c.sumAndUpdateResult();
    return c.getResult();
}
```



25

2.2. Common Coupling (Móc nối thành phần chung)



- ❖ Định nghĩa: Các module cùng chia sẻ chung định dạng dữ liệu, giao thức truyền thông, ...

- ❖ Không tốt vì

- khó hiểu
- khó xác định component nào ảnh hưởng tới dữ liệu nào
- khó tái sử dụng component
- khó điều khiển truy cập tới dữ liệu, ...



26

Ví dụ

- ❖ Tất cả thành phần cùng truy cập biến toàn cục, 1 số thiết lập giá trị, 1 số đọc giá trị
- ❖ OOP: biến static để là public
- ❖ Bất lợi khi debug
 - Biến toàn cục nhận giá trị khó hiểu
 - Xử lý đồng bộ
 - Khó theo dõi



27

Ví dụ

```
while( global_variable > 0 ) {
    switch( global_variable ) {
        case 1: function_a(); break;
        case 2: function_b(); break;
        ...
        case n:...
    }
    global_variable++;
}
```



28

2.3. Control Coupling (Móc nối điều khiển)

❖ Định nghĩa: Component truyền tham số điều khiển tới component khác

- Ví dụ, một method làm nhiều nhiệm vụ, tùy theo tham số flag truyền vào.

❖ Có thể tốt/xấu, tùy tình huống

- Xấu nếu component cần biết cấu trúc bên trong và cách xử lý của component kia.
- Tốt nếu tham số giúp ích cho việc factoring và tái sử dụng các chức năng

❖ Ví dụ tốt: hàm sắp xếp mảng, nhận tham số là hàm so sánh 2 phần tử.

Ví dụ 1

❖ Phương thức

- updateCustomer(int whatKind, Customer customer) trong đó
 - whatKind nhận giá trị ADD, EDIT hoặc DELETE, và
 - customer được dùng cho EDIT, nhưng không dùng gì trong ADD, chỉ trường id dùng cho DELETE.

Ví dụ 1 – giải pháp

❖ Cần tránh, vì module gọi phải biết logic của module được gọi

❖ Phương thức sẽ khó hiểu, khó bảo trì

❖ Để tránh: cần tạo các phương thức riêng cho từng chức năng. Cần áp dụng kế thừa, đa hình.

Ví dụ 2 – tight coupling

```
public void run() {
    takeAction(1);
}

public void takeAction(int key) {
    switch (key) {
        case 1:
            System.out.println("ONE RECEIVED");
            break;
        case 2:
            System.out.println("TWO RECEIVED");
            break;
    }
}
```

Ví dụ 2 – loose coupling

```
public void run() {
    Printable printable = new PrinterOne();
    takeAction(printable);
}
public void takeAction(Printable printable) {
    printable.print();
}

public interface Printable {
    void print();
}

public class PrinterOne implements Printable {
    @Override
    public void print() {
        System.out.println("ONE RECEIVED");
    }
}

public class PrinterTwo implements Printable {
    @Override
    public void print() {
        System.out.println("TWO RECEIVED");
    }
}
```

33

Ví dụ 3 – Vấn đề?

```
int testStack(int kind, Stack S) {
    if (((kind == 1) && (S.num == 0)) || ((kind == 2) && (S.num == MAX)))
        return 1;
    else
        return 0;
}
```

34

Ví dụ 3 – Giải pháp

❖ Nên có 2 hàm rõ ràng

```
int emptyStack(Stack S) {
    if (S.num == 0)
        return 1;
    else
        return 0;
}

int fullStack(Stack S) {
    if (S.num == MAXSTACK)
        return 1;
    else
        return 0;
}
```

35

2.4. Stamp Coupling (Móc nối dữ liệu phức hợp)

- ❖ Định nghĩa: Tham số dữ liệu được truyền trong cấu trúc dữ liệu có nhiều trường thông tin, nhưng chỉ dùng một số trường trong đó
- ❖ Có thể làm như vậy, nhưng cần có lý giải rõ ràng, thay vì lý do là "lười"
- ❖ Hai giải pháp: 1-dùng interface, 2-truyền tham số là dữ liệu nguyên thủy

36

Ví dụ 1

- ❖ Tính thuế phải trả của nhân viên

```
int incomeTaxPayable(Person p) {  
    /* code which refers to only p.salary */  
}
```

- ❖ Hàm incomeTaxPayable này làm LTV khác nghĩ nó dùng tất cả các trường đối tượng person → debug khó hiểu.

Ví dụ 1 – giải pháp

```
int incomeTaxPayable(int salary) {  
    /* code which refers salary */  
}
```

Ví dụ 2

```
public class Emailer {  
    public void sendEmail(Employee e, String text) {  
        ...  
    }  
    ...  
}
```

- ❖ Giải pháp Sử dụng dữ liệu nguyên thủy:

```
public class Emailer {  
    public void sendEmail(String name, String email, String text) {  
        ...  
    }  
    ...  
}
```

Ví dụ 2

- ❖ Giải pháp sử dụng interface

```
public interface Addressee {  
    public abstract String getName();  
    public abstract String getEmail();  
}  
  
public class Employee implements Addressee {...}  
  
public class Emailer {  
    public void sendEmail(Addressee e, String text) {  
        ...  
    }  
    ...  
}
```

2.5. Data coupling (Móc nối dữ liệu nguyên thủy)

❖ Định nghĩa: Module này gọi module kia, chỉ dùng tham số nguyên thủy thay vì đối tượng

- Càng nhiều tham số, tính coupling càng cao
- Nên bỏ bớt các tham số nếu được

41

Data coupling

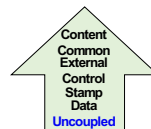
- ❖ Điểm mạnh
 - là loại coupling thấp nhất → tốt nhất.
 - 1 module chỉ được cung cấp các phần tử dữ liệu nó cần.
- ❖ Điểm yếu
 - module sẽ có nhiều tham số truyền vào
- ❖ Có mâu thuẫn (trade-off) giữa data coupling và stamp coupling

42

42

Uncoupled

❖ Các thành phần không liên quan tới nhau



43

43

Nội dung

1. Cách thức thiết kế
2. Móc nối linh hoạt (low coupling)
3. Tính kết dính cao (high cohesion)

44

44

3. Cohesion

- ❖ Định nghĩa 1: Mức độ cùng hướng tới mục tiêu chung giữa các phần tử trong 1 component
- ❖ Định nghĩa 2: Cohesion: Mỗi phần tử có 1 chức năng rõ ràng
- ❖ Là “chất keo” tạo nên 1 component
- ❖ Cao là tốt

Cohesion

- ❖ Mỗi module một nhiệm vụ, không nhiều hơn một, nhiệm vụ rõ ràng → high cohesion
- ❖ Một module nhiều nhiệm vụ → không có nhiệm vụ rõ ràng, khó đặt tên → low cohesion
- ❖ Nguyên lý **Single Responsibility Principle** (SOLID) hướng tới các lớp có tính cohesion cao
- ❖ Cohesion tang nếu
 - Các tính năng trong lớp cung cấp qua các phương thức có nhiều điểm chung going nhau
 - Mỗi phương thức thực thi các hoạt động liên quan tới nhau, thay vì các hoạt động, dữ liệu không liên quan

Lợi ích của tính kết dính cao

- ❖ Giảm độ phức tạp
- ❖ Module ít chức năng → đơn giản, dễ hiểu, dễ bảo trì, ít ảnh hưởng tới nhau
- ❖ Mỗi module 1 chức năng → Tăng tính tái sử dụng module.

Ví dụ

```
class A
checkEmail()
validateEmail()
sendEmail()
printLetter()
printAddress()
```

Fig: Low cohesion

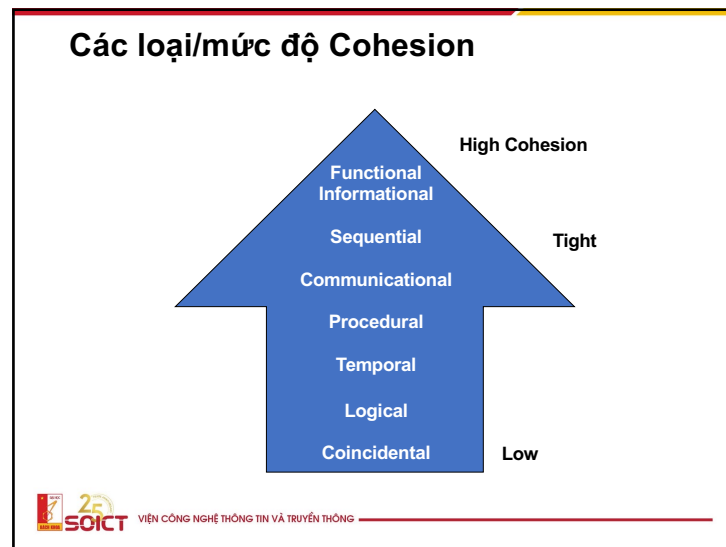
```
class A
checkEmail()
```

```
class B
validateEmail()
```

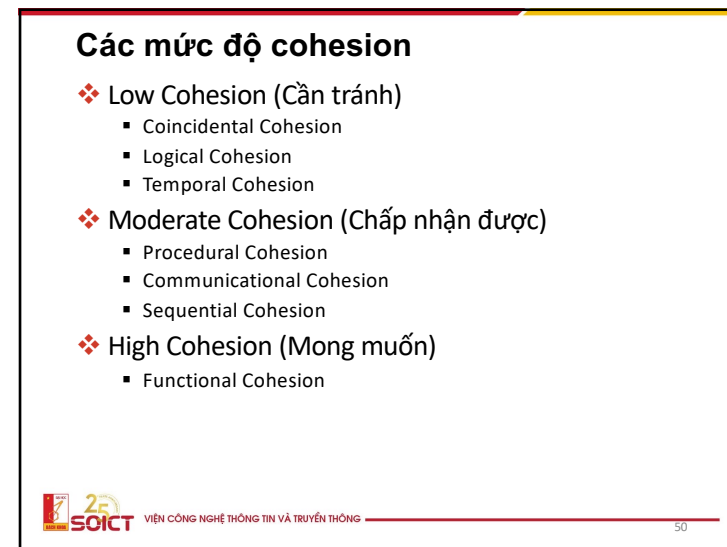
```
class C
sendEmail()
```

```
class D
printLetter()
```

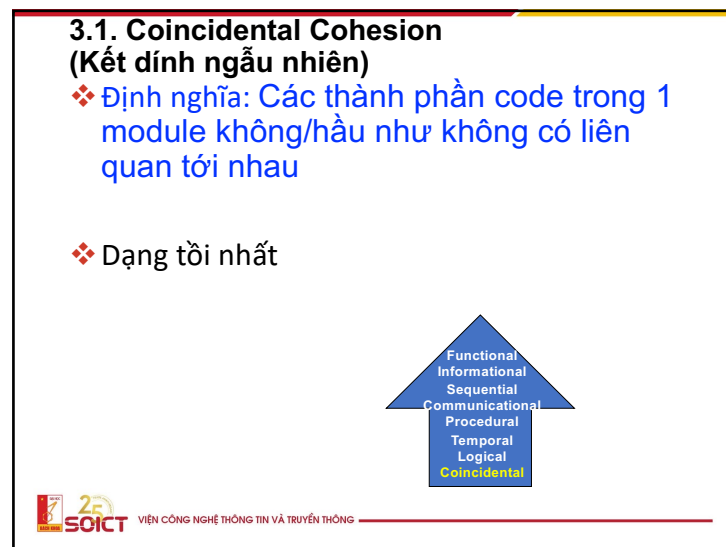
Fig: High cohesion



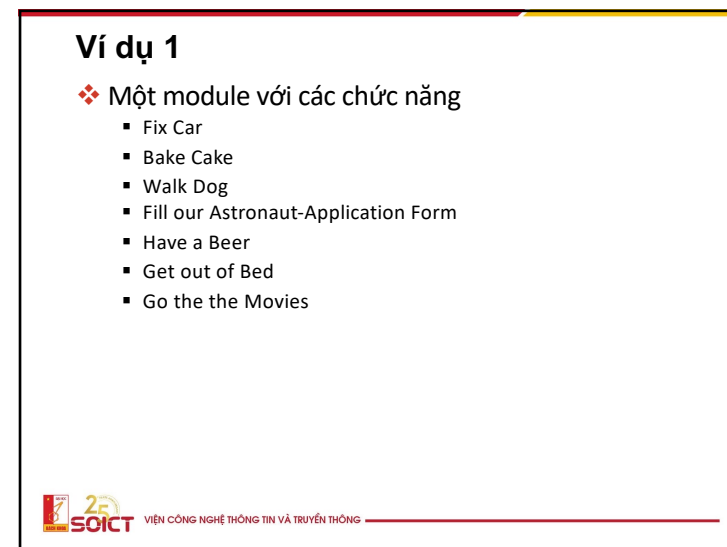
49



50



51



52

Ví dụ 2

```
class Joe {  
    // converts a path in windows to one in linux  
    public String win2lin(String);  
  
    // number of days since the beginning of time  
    public int days(String);  
  
    // outputs a financial report  
    public void outputReport(FinanceData);  
}
```

53

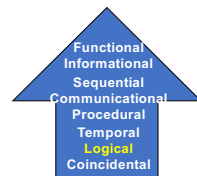
Ví dụ 3

```
class Output {  
    // outputs a financial report  
    public void outputReport(FinanceData);  
  
    // outputs the current weather  
    public void outputWeather(WeatherData);  
  
    // output a number in a nice formatted way  
    public void outputInt(int);  
}
```

54

3.2. Logical Cohesion (Kết dính logic)

- ❖ Định nghĩa: Các thành phần trong 1 component có liên quan tới nhau về mặt logic, không phải về mặt chức năng
- ❖ Phần tử sử dụng các component này sẽ lựa chọn một component nào đó để sử dụng.



55

Ví dụ 1

- ❖ lên kế hoạch đi du lịch, chọn 1 trong các hình thức sau đến đến điểm cần đến
 - Go by Car
 - Go by Train
 - Go by Boat
 - Go by Plane
- ❖ Các hoạt động này liên quan gì tới nhau mà đặt trong cùng 1 module? Đó là các hình thức đi lại. Nhưng khi đi du lịch, ta chỉ chọn 1 hình thức, không chọn nhiều

56

Ví dụ 2

- ❖ Hàm in báo cáo, tùy tham số flag truyền vào, hàm sẽ xử lý hình thức báo cáo tương ứng.
 - Printing a local sales report.
 - Printing a regional sales report.
 - Printing a national sales report.
- ❖ Ta chỉ chọn 1 trong các hình thức báo cáo này → không nên cài đặt tất cả các hình thức báo cáo trong hàm đó

57

Ví dụ 3

- ❖ Cần đọc dữ liệu vào, từ nhiều nguồn khác nhau.
- ❖ Cho tất cả code vào 1 component. Truyền vào tham số flag (0: tape, 1: disk, 2: network).
- ❖ Component ở đây có thể là method/class

```
public class Example {  
    public void readDataFromTape(Tape t) {  
    }  
    public void readDataFromDisk(Disk d) {  
    }  
    public void readDataFromNetwork(Network n) {  
    }  
}
```

58

58

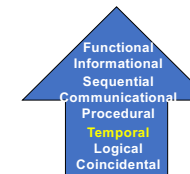
Ví dụ 3 – giải pháp

```
public abstract class DataReader<T> {  
    public void read(T t)  
}  
  
public class TapeDataReader extends DataReader<Tape> {  
    @Override  
    public void read(Tape t) {  
    }  
}  
  
public class DiskDataReader extends DataReader<Disk> {  
    @Override  
    public void read(Disk d) {  
    }  
}  
  
public class NetWorkDataReader extends DataReader<Network> {  
    @Override  
    public void read(Network n) {  
    }  
}
```

59

3.3. Temporal Cohesion (Kết dính hướng thời gian)

- ❖ Định nghĩa: Các phần tử trong một component liên kết tới nhau về mặt thời gian: cùng được thực hiện ở một thời điểm nào đó
- Khó sửa đổi, vì phức tạp. Đồng thời Khó tái sử dụng



60

Ví dụ 1

❖ Các hoạt động sau được đặt trong cùng 1 module, vì đều được thực hiện vào buổi tối

- Put out Milk Bottles
- Put out Cat
- Turn off TV
- Brush Teeth

61

Ví dụ 2

❖ Nhiều công việc khởi tạo các thành phần hệ thống được đưa hết vào lớp Init

```
class Init {  
    // initializes financial report  
    public void initReport(FinanceData);  
  
    // initializes current weather  
    public void initWeather(WeatherData);  
  
    // initializes master count  
    public void initCount();  
}
```

62

Ví dụ 2 – giải pháp

❖ Giải pháp: Dùng đa hình, interface/abstract class

❖ Lưu ý: Việc khởi tạo của từng lớp cần được tách độc lập với nhau, do lớp đó đảm nhận

Remember: The goal is to produce procedures which can do their single function independently of other procedures.

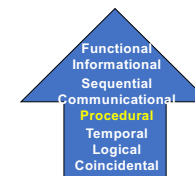
63

3.4. Procedural Cohesion (Kết dính thủ tục)

❖ Định nghĩa: Các thành phần được gom lại vì chúng cần được thực hiện theo thứ tự

❖ Vẫn thiếu tính kết dính và khó tái sử dụng

❖ Lưu ý: Không cần dữ liệu ra của bước này làm dữ liệu vào của bước sau (sequential cohesion)



64

Ví dụ 1

❖ A *Prepare for Holiday Meal* module:

- Clean Utensils from Previous Meal
- Prepare Turkey for Roasting
- Make Phone Call
- Take Shower
- Chop Vegetables
- Set Table

→ Cần tách các component thành các module khác nhau, đặt ở vị trí phù hợp. Không phải vì được thực hiện theo trình tự ở 1 xử lý nào đó mà đặt trong cùng 1 component.

Ví dụ 2

```
public class Example {  
    public void readData() {}  
    public void sendEmail() {}  
}
```



```
public class DataReader {  
    public void readData() {}  
}
```

```
public class Email {  
    public void sendEmail() {}  
}
```

Procedural cohesion

❖ Lợi ích

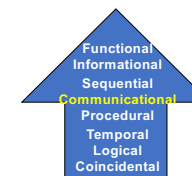
- Các phần tử liên quan tới nhau (về trình tự thực hiện) trong 1 được đặt cạnh nhau

❖ Tác hại

- Chưa tự nhiên. Các phần tử liên quan tới nhau vì trình tự thực hiện, chỉ trong procedure đó, thay vì thực sự có liên quan tới nhau
- Có thể gây ảnh hưởng lan truyền khi sửa 1 phần tử

3.5. Communicational Cohesion (Kết dính giao tiếp)

- ❖ Định nghĩa: Các thành phần con được gom lại trong 1 thành phần lớn, vì các thành phần con cùng thao tác trên một cấu trúc dữ liệu



Ví dụ

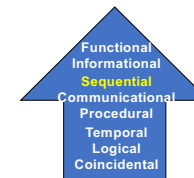
```
public class Example {  
    private Transaction trans;  
  
    public void readTransaction() {}  
    public void sortTransaction() {}  
    public void calculateTransactionMean() {}  
    public void printTransaction() {}  
    public void saveTransaction() {}  
}
```



69

3.6. Sequential Cohesion (Kết dính tuần tự)

- ❖ Tương tự như procedural. Khác biệt: bổ sung thêm yêu cầu input của phần tử này là output của phần tử trước đó
- ❖ Là loại cohesion tốt



70

Ví dụ

- ❖ Module với các hoạt động
 - Clean Car Body
 - Fill in Holes in Car
 - Sand Car Body
 - Apply Primer
- ❖ Tham số car sẽ được truyền từ tác vụ trước sang tác vụ sau



71

Sequential cohesion

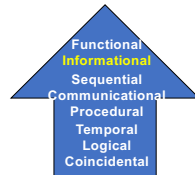
- ❖ Ví dụ: tính toán quỹ đạo và in ra kết quả. Cần phải làm theo thứ tự. Quỹ đạo tính toán được sẽ được in ra trong kết quả
- ❖ Lợi ích
 - tách biệt chức năng rõ ràng hơn so với loại communicational cohesion
- ❖ Bất lợi
 - hạn chế tái sử dụng. VD khi cần tính quỹ đạo rồi ghi kết quả ra file/truyền qua mạng, ..., sẽ phải tạo module mới. Module mới và module cũ bị lặp phần code tính toán quỹ đạo, chỉ khác biệt phần xử lý sau



72

3.7. Informational Cohesion (Kết dính thông tin)

- ❖ **Định nghĩa:** Cũng thao tác trên dữ liệu chung như communicational cohesion. Nhưng mỗi thao tác là độc lập với nhau.



73

Eví dụ 1

- ❖ 3 phương thức này hoạt động độc lập với nhau, không phụ thuộc vào nhau, cùng thao tác trên 1 CTDL EmpRec

```
void employeeAdd(int status, EmpRec rec);
```

```
void employeeUpdate(int status, EmpRec rec);
```

```
void employeeDelete(int status, EmpRec rec);
```

74

74

Ví dụ 2

```
public class Example {  
    public Transaction readTransaction() {}  
    public void sortTransaction(Transaction trans) {}  
    public void calculateTransactionMean(Transaction trans) {}  
    public void printTransaction(Transaction trans) {}  
    public void saveTransaction(Transaction trans) {}  
}
```

75

75

Informational Cohesion

- ❖ **Lợi ích**
 - đóng gói được các chức năng liên quan tới dữ liệu được cung cấp, giúp dễ hiểu, dễ đọc code, giảm chi phí bảo trì
- ❖ **Bất lợi**
 - Nếu ứng dụng nào đó không dùng tất cả các chức năng → lãng phí bộ nhớ

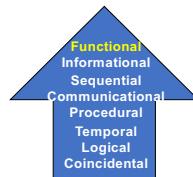
76

76

3.8. Functional Cohesion (Kết dính chức năng)

❖ Định nghĩa: Tất cả các thành phần trong 1 module cùng hướng đến 1 chức năng duy nhất mà module cung cấp

❖ Là loại lý tưởng nhất



3.8. Functional Cohesion

❖ Lợi ích

- tăng tính tái sử dụng, tách biệt chức năng rõ ràng, mỗi module 1 chức năng, giúp dễ hiểu, dễ sửa đổi, giảm chi phí bảo trì

❖ Bất lợi

- Tăng số lượng module

Khác biệt giữa Cohesion và Coupling

Cohesion

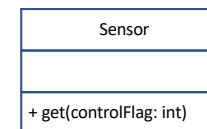
- ❖ Cohesion is the indication of the relationship within module
- ❖ Cohesion shows the module's relative functional strength
- ❖ Cohesion is a degree (quality) to which a component / module focuses on the single thing
- ❖ While designing we should strive for high cohesion. Ex: cohesive component/module focus on a single task with little interaction with other modules of the system
- ❖ Cohesion is the kind of natural extension of data hiding, for example, class having all members visible with a package having default visibility
- ❖ Cohesion is Intra-Module Concept

Coupling

- ❖ Coupling is the indication of the relationships between modules
- ❖ Coupling shows the relative independence among the modules
- ❖ Coupling is a degree to which a component / module is connected to the other modules
- ❖ While designing we should strive for low coupling. Ex: dependency between modules should be less
- ❖ Making private fields, private methods and non public classes provides loose coupling
- ❖ Coupling is Inter-Module Concept

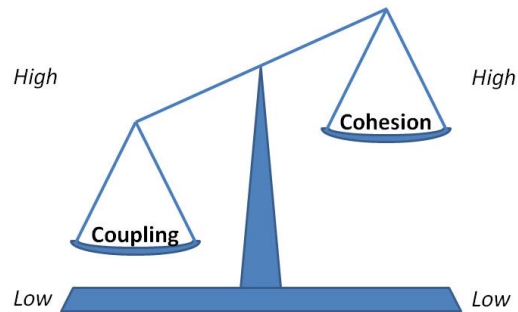
Ví dụ

Phân tích và Cải tiến code???



```
public void get (int controlFlag){
    switch (controlFlag) {
        case 0:
            return this.humidity;
            break;
        case 1:
            return this.temperature;
            break;
        default:
            throw new UnknownFlagException
    }
}
```

Quan hệ giữa Coupling và Cohesion?



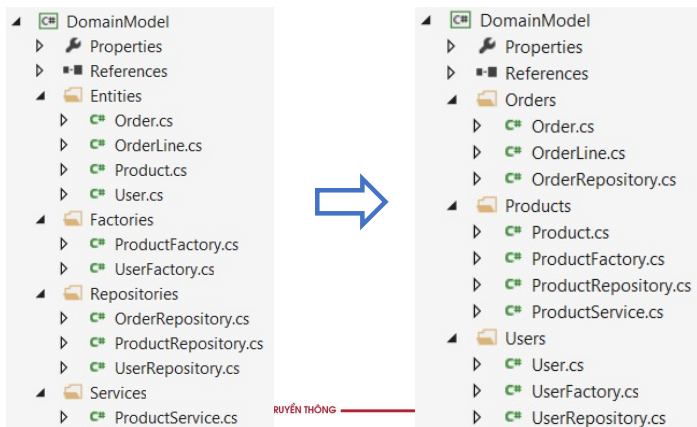
82

Cohesion và Coupling

- ❖ Thường là quan hệ tradeoff. Vì độ phức tạp sẽ hoặc đi vào 1 module, hoặc dàn trải ra giữa các module.
 - 1 module đơn giản sẽ có tính high cohesion. Nhưng khi đó, nó phải phụ thuộc vào nhiều module khác, phải phối hợp với nhiều module khác → tăng coupling
 - Kết nối giữa các module đơn giản để đảm bảo tính low coupling → module phải có nhiều nhiệm vụ hơn → làm giảm tính cohesion

83

Ví dụ



84

 **SOICT** 25
ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Bài 10: Các nguyên lý thiết kế

1

1

Nội dung

- ❖ 1. Nguyên lý GRASP
- ❖ 2. Nguyên lý tri thức tối thiểu
- ❖ 3. Nguyên lý SOLID

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

2

2

Nội dung

- ❖ 1. Nguyên lý GRASP
- ❖ 2. Nguyên lý tri thức tối thiểu
- ❖ 3. Nguyên lý SOLID

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

3

3

GRASP

- ❖ GRASP - General Responsibility Assignment Software Patterns (or Principles): Nguyên lý/mẫu thiết kế phần mềm phân định trách nhiệm
- ❖ GRASP: tất cả các nguyên lý GRASP đều hướng tới trọng tâm là phân trách nhiệm cho ai
- ❖ GRASP không liên quan tới SOLID

 **SOICT** 25
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

4

4

GRASP

Có 9 mẫu (nguyên lý) sử dụng trong GRASP

1. Information expert
2. Creator
3. Controller
4. Low coupling
5. High cohesion
6. Indirection
7. Polymorphism
8. Pure fabrication
9. Protected variations- Don't Talk to Strangers

Craig Larman: *Apply UML and Patterns – An Introduction to Object-Oriented Analysis and Design*
SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

5

5

Chuyên gia thông tin

- ❖ Cho đối tượng o, những nhiệm vụ nào có thể được giao cho o?
- ❖ Nguyên lý **chuyên gia thông tin**: Trao nhiệm vụ cho lớp có đầy đủ thông tin để hoàn thành nhiệm vụ đó.
- ❖ Lớp có thể đã có sẵn thông tin để trả lời. Hoặc lớp có thể trao đổi với các lớp khác để lấy đủ thông tin cần thiết để thực hiện nhiệm vụ

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

6

6

Ví dụ

- ❖ Cần lấy tất cả các videos trong VideoStore.
- ❖ Vì VideoStore biết về tất cả videos, trách nhiệm lấy tất cả các videos trong VideoStore được giao cho lớp VideoStore.
- ❖ VideoStore là lớp chuyên gia thông tin

SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

7

7

Ví dụ

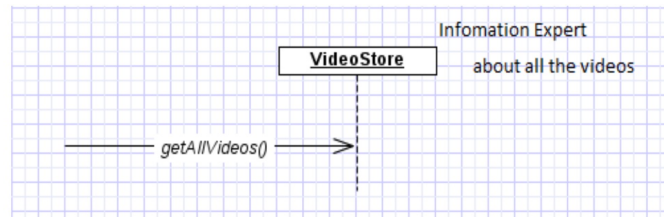


SOICT VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

8

8

Ví dụ



9

Khởi tạo

- ❖ Ai sẽ chịu trách nhiệm khởi tạo một đối tượng?
- ❖ Quyết định dựa trên quan hệ và tương tác giữa các đối tượng
- ❖ B sẽ khởi tạo A nếu:
 - B chứa A
 - B lưu lại A
 - B dùng A như thành phần thiết yếu
 - B khởi tạo dữ liệu cho A

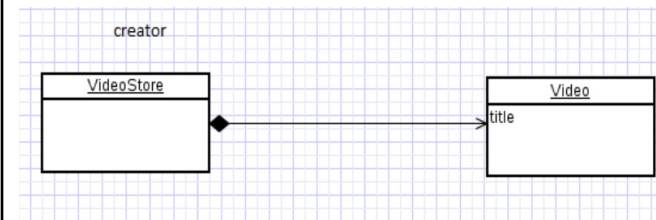
10

Ví dụ

- ❖ Xét ví dụ với VideoStore và Video
- ❖ VideoStore có quan hệ kết tập với Video. VideoStore chứa các Video
- ❖ Do đó, ta có thể khởi tạo đối tượng Video trong lớp VideoStore

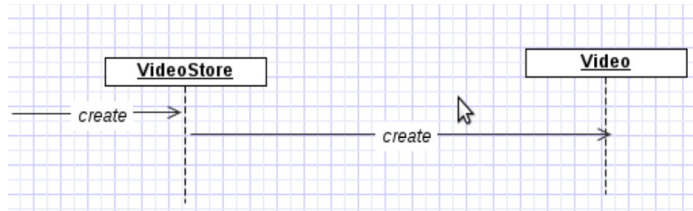
11

Ví dụ



12

Ví dụ



13

Điều khiển

- ❖ Đối tượng nào sẽ chịu trách nhiệm chuyển request từ các đối tượng UI tới các đối tượng nghiệp vụ
- ❖ Khi một yêu cầu đến từ đối tượng tầng UI, mẫu điều khiển giúp chúng ta quyết định đối tượng đầu tiên nhận yêu cầu này là đối tượng nào
- ❖ Đối tượng điều khiển - controller object: nhận yêu cầu từ tầng UI, và điều phối các đối tượng khác ở tầng nghiệp vụ việc thực hiện công việc (Đôi khi phải cần nhiều đối tượng phối hợp thực hiện 1 công việc)



14

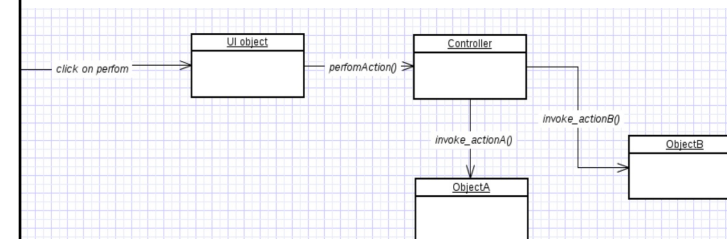
Điều khiển

- ❖ Một đối tượng sẽ là đối tượng controller nếu
 - Đối tượng đại diện cho toàn bộ hệ thống (facade controller)
 - Đối tượng đại diện cho 1 nghiệp vụ use case, xử lý một chuỗi các thao tác (use case or session controller).
- ❖ Lợi ích
 - Có thể tái sử dụng lớp controller
 - Dễ quản lý trạng thái trong use case
 - Có thể điều khiển chuỗi các hoạt động



15

Ví dụ



16

Điều khiển quá tải - Bloated Controllers

- ❖ Lớp Controller là quá tải - bloated, nếu
 - Lớp có quá nhiều trách nhiệm.
 - Giải pháp – thêm các controllers khác
 - Lớp điều khiển xử lý luôn quá nhiều nhiệm vụ thay vì chuyển các nhiệm vụ đó cho các lớp khác
 - Giải pháp – chuyển nhiệm vụ cho các lớp khác

17

Kết nối lỏng lẻo - Low Coupling

- ❖ Mức độ các đối tượng liên kết với nhau?
- ❖ Coupling – một đối tượng phụ thuộc vào các đối tượng khác.
- ❖ Khi một đối tượng thay đổi, sẽ ảnh hưởng đến các đối tượng phụ thuộc.
- ❖ Kết nối lỏng lẻo - Giảm tác động của thay đổi.
- ❖ Kết nối lỏng lẻo - phân công trách nhiệm để đảm bảo kết nối lỏng lẻo .
- ❖ Giảm thiểu sự phụ thuộc do đó giúp hệ thống dễ bảo trì, hiệu quả và dễ tái sử dụng mã nguồn

18

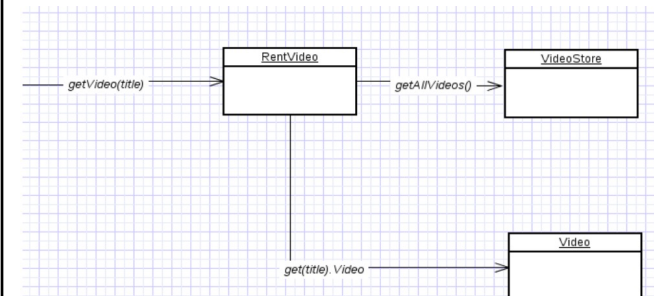
Low coupling

- ❖ 2 lớp là phụ thuộc nếu
 - lớp này liên kết với lớp khác
 - lớp này kế thừa lớp khác Two elements are coupled, if

19

Ví dụ

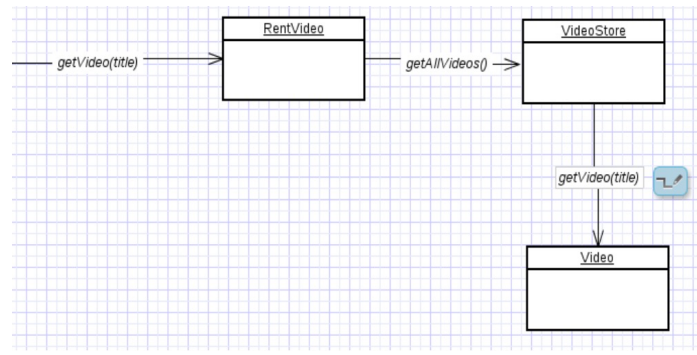
- ❖ RentVideo phụ thuộc vào cả VideoStore và Video
→ high coupling



20

Ví dụ

- ❖ VideoStore và Video phụ thuộc vào nhau, Rent phụ thuộc vào VideoStore. Do đó, kết nối là lỏng lẻo hơn (low coupling)



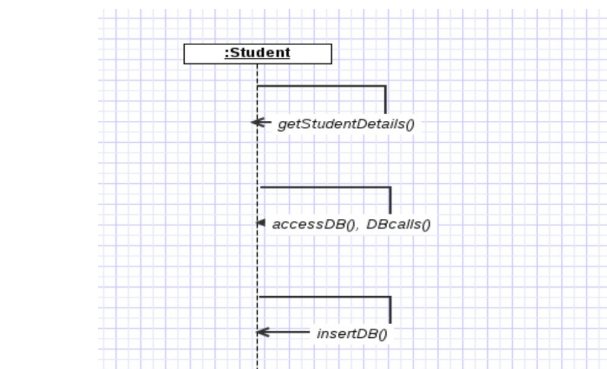
21

Tính nhất quán cao - High Cohesion

- ❖ Mức độ liên quan giữa các operation trong một lớp?
- ❖ Các chức năng liên quan tới nhau trong một đơn vị quản lý
- ❖ Cần đảm bảo tính high cohesion
- ❖ Xác định rõ ràng nhiệm vụ của phần tử
- ❖ Lợi ích
 - Dễ hiểu dễ bảo trì
 - Tăng tính tái sử dụng mã nguồn
 - Góp phần làm giảm tính coupling

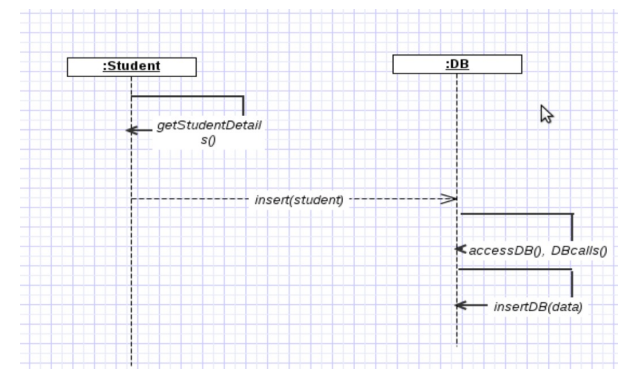
22

Ví dụ



23

Ví dụ



24

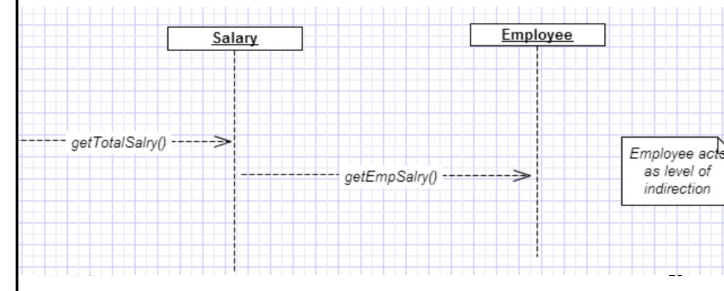
Gián tiếp - Indirection

- ❖ Cách thức tránh phụ thuộc trực tiếp giữa các phần tử?
- ❖ Indirection sẽ đưa ra một đơn vị trung gian để thực hiện giao tiếp giữa phần tử, do đó các phần tử không bị trực tiếp phụ thuộc vào nhau
- ❖ Lợi ích: low coupling
- ❖ Ví dụ: Adapter, Facade, Observer

25

Ví dụ

- ❖ Sử dụng đa hình với Employee là phần tử trung gian



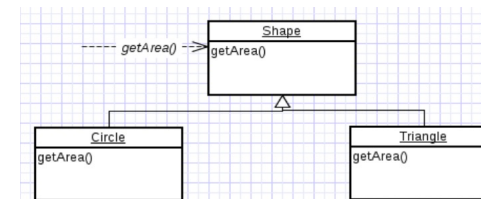
26

Đa hình - Polymorphism

- ❖ Khi muốn hành vi khác nhau tùy theo kiểu cụ thể của đối tượng thì làm thế nào?
- ❖ Sử dụng đa hình
- ❖ Lợi ích: xử lý đơn giản và dễ dàng

27

Ví dụ



28

Pure Fabrication

- ❖ Vấn đề: phân trách nhiệm cho lớp nào, khi mà áp dụng các nguyên lý ở trên lại dẫn đến vi phạm nguyên lý high cohesion và low coupling
- ❖ Giải pháp: tạo một lớp riêng, độc lập và gán trách nhiệm cho lớp đó. Lớp này thường gọi là lớp tiện ích/lớp dịch vụ.



29

Ví dụ 1

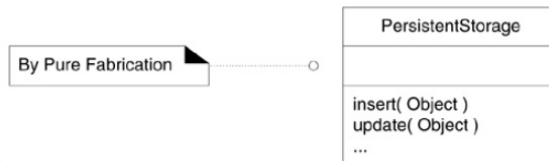
- ❖ Cần lưu đối tượng lớp **Sale** vào CSDL. Theo nguyên lý **Information Expert**, nhiệm vụ lưu được gán cho chính lớp **Sale**, vì lớp này có tất cả dữ liệu cần lưu. Nhưng
 - Sale có nhiều nhiệm vụ → không đảm bảo tính high cohesion
 - Sale phụ thuộc vào các lớp tiện ích để lưu DB → tăng tính coupling
 - Thao tác save còn lặp lại nhiều lần với các đối tượng khác (như lớp Customer) → không có tính tái sử dụng



30

Ví dụ 1

- ❖ Giải pháp: tạo một lớp mới (PersistentStorage) chịu trách nhiệm lưu trữ đối tượng Sale vào CSDL
- ❖ Lợi ích:
 - Lớp Sale vẫn đảm bảo có thiết kế tốt, có tính high cohesion và low coupling.
 - Lớp PersistentStorage có tính cohesion khá cao, nhiệm vụ là lưu trữ/thao tác với CSDL
 - Lớp PersistentStorage tổng quát, có tính tái sử dụng cao



31

Ví dụ 2

```
interface IForeignExchange {
    List<ConversionRate> getConversionRates();
}

class ConversionRate {
    private String from;
    private String to;
    private double rate;
    public ConversionRate(String from, String to, double rate) {
        this.from = from;
        this.to = to;
        this.rate = rate;
    }
}

public class ForeignExchange implements IForeignExchange {
    public List<ConversionRate> getConversionRates() {
        List<ConversionRate> rates = ForeignExchange.getConversionRatesFromExternalApi();
        return rates;
    }

    private static List<ConversionRate> getConversionRatesFromExternalApi() {
        // Communication with external API. Here is only mock.
        List<ConversionRate> conversionRates = new ArrayList<ConversionRate>();
        conversionRates.add(new ConversionRate("USD", "EUR", 0.88));
        conversionRates.add(new ConversionRate("EUR", "USD", 1.13));
        return conversionRates;
    }
}
```



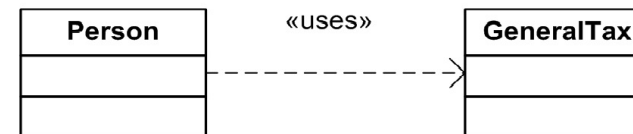
32

Thích ứng với các thay đổi - Protected Variation

- ❖ Vấn đề: Thiết kế 1 thành phần ntn để khi thành phần đó thay đổi, ít gây ảnh hưởng nhất không mong muốn nhất tới các thành phần khác
- ❖ Giải pháp: Xác định các điểm tương lai sẽ có sự thay đổi/không ổn định, tạo interface tương ứng bọc lại

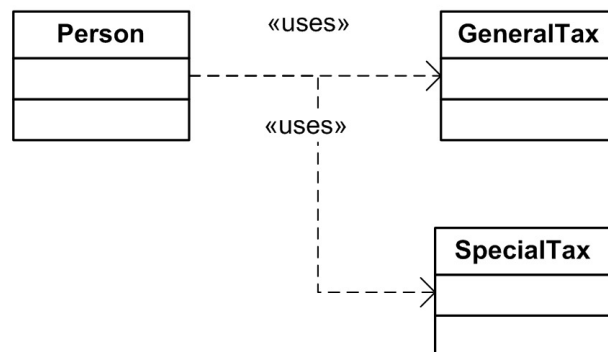
33

Ví dụ (1/3)



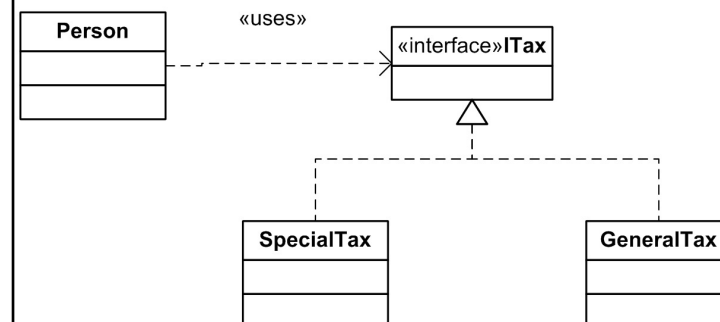
34

Ví dụ (2/3)



35

Ví dụ (3/3)



36

Nội dung

- ❖ 1. Nguyên lý GRASP
- ❖ **2. Nguyên lý tri thức tối thiểu**
- ❖ 3. Nguyên lý SOLID

Nguyên lý Demeter - nguyên lý tri thức tối thiểu (Karl Lieberherr)

- ❖ Một đối tượng khi tương tác với các đối tượng khác, chỉ nên biết ít nhất có thể về nội bộ cấu trúc của các đối tượng đó (giúp đảm bảo tính low cohesion)
 - Ý tưởng ... “chỉ nói chuyện với người bạn trực tiếp của bạn”
- ❖ Ví dụ code tồi:

```
general.getColonel().getMajor(m).getCaptain(cap)
.getSergeant(ser).getPrivate(name).digFoxHole();
```
- ❖ Ví dụ code tốt

```
general.superviseFoxHole(m, cap, ser, name);
```

Nguyên lý tri thức tối thiểu

- ❖ Trong phương thức M của đối tượng O chỉ được gọi các phương thức của các loại đối tượng sau:
 - Chính đối tượng O
 - Các đối tượng trong tham số của M
 - Các đối tượng được tạo/khởi tạo trong M
 - Các đối tượng thuộc tính của O

Lưu ý: không cần máy móc tuân thủ trong mọi trường hợp! Nhưng cần cân nhắc, sẽ có thiết kế tốt hơn

Không đảm bảo nguyên lý tri thức tối thiểu

- ```
objectA.getObjectB().getObjectC().doSomething();
```
- ❖ objectA về sau có thể sẽ không còn tham chiếu tới ObjectB nữa.
  - ❖ Phương thức doSomething() trong ObjectC có thể sẽ không còn tồn tại nữa.
  - ❖ Sẽ gặp các loại lỗi như NullPointerException, NoMethodError nếu như ObjectB và ObjectC bị null.
  - ❖ Khi đóng gói objectA để tái sử dụng, bạn sẽ cần phải kèm ObjectB, ObjectC với nó => Sự phụ thuộc lẫn nhau giữa các thành phần trong hệ thống tăng cao. (tightly coupled)

## Lợi ích

- ❖ Class sẽ loosely coupled hơn, những thành phần trong hệ thống sẽ ít phụ thuộc nhau hơn.
- ❖ Đóng gói và tái sử dụng sẽ dễ dàng hơn.
- ❖ Việc test sẽ dễ hơn nhiều, phần setup của test sẽ đơn giản hơn.
- ❖ Ít lỗi hơn.



## Ví dụ 1

```
public class Customer {
 private String firstName;
 private String lastName;
 private Wallet myWallet;

 public String getFirstName(){
 return firstName;
 }
 public String getLastName(){
 return lastName;
 }
 public Wallet getWallet(){
 return myWallet;
 }
}
```



## Ví dụ 1

```
public class Wallet {
 private float value;
 public float getTotalMoney() {
 return value;
 }
 public void setTotalMoney(float newValue) {
 value = newValue;
 }
 public void addMoney(float deposit) {
 value += deposit;
 }
 public void subtractMoney(float debit) {
 value -= debit;
 }
}
```



## Ví dụ 1

```
// code from some method inside the Paperboy class...
payment = 2.00; // "I want my two dollars!"
Wallet theWallet = myCustomer.getWallet();
if (theWallet.getTotalMoney() > payment) {
 theWallet.subtractMoney(payment);
} else {
 // come back later and get my money
}
```

*Is this Bad? Why?*



## Ví dụ 1 – Cải tiến

```
public class Customer {
 private String firstName;
 private String lastName;
 private Wallet myWallet;
 public String getFirstName(){
 return firstName;
 }
 public String getLastName(){
 return lastName;
 }
 public float getPayment(float bill) {
 if (myWallet != null) {
 if (myWallet.getTotalMoney() > bill) {
 theWallet.subtractMoney(payment);
 return payment;
 }
 }
 }
}
```

45

## Ví dụ 1 – Cải tiến

```
// code from some method inside the Paperboy class...
payment = 2.00; // "I want my two dollars!"
paidAmount = myCustomer.getPayment(payment);
if (paidAmount == payment) {
 // say thank you and give customer a receipt
} else {
 // come back later and get my money
}
```

*Why Is This Better?*

46

## Ví dụ 2

```
public class Band {
 private Singer singer;
 private Drummer drummer;
 private Guitarist guitarist;
}
```

47

## Ví dụ 2

```
class TourPromoter {
 public String makePosterText(Band band) {
 String guitaristsName = band.getGuitarist().getName();
 String drummersName = band.getDrummer().getName();
 String singersName = band.getSinger().getName();
 StringBuilder posterText = new StringBuilder();

 posterText.append(band.getName())
 posterText.append(" featuring: ");
 posterText.append(guitaristsName);
 posterText.append(", ");
 posterText.append(singersName);
 posterText.append(", ")
 posterText.append(drummersName);
 posterText.append(", ")
 posterText.append("Tickets £50.");

 return posterText.toString();
 }
}
```

48

## Ví dụ 2 – Cải tiến

```
public class Band {
 private Singer singer;
 private Drummer drummer;
 private Guitarist guitarist;

 public String[] getMembers() {
 return {
 singer.getName(),
 drummer.getName(),
 guitarist.getName()
 };
 }
}
```

49

## Ví dụ 2 – Cải tiến

```
public class TourPromoter {
 public String makePosterText(Band band) {
 StringBuilder posterText = new StringBuilder();

 posterText.append(band.getName());
 posterText.append(" featuring: ");
 for(String member: band.getMembers()) {
 posterText.append(member);
 posterText.append(", ");
 }
 posterText.append("Tickets: £50");

 return posterText.toString();
 }
}
```

50

## Nội dung

- ❖ 1. Nguyên lý GRASP
- ❖ 2. Nguyên lý tri thức tối thiểu
- ❖ **3. Nguyên lý SOLID**

51

## SOLID

*“Principles Of OOD”, Robert C. Martin (“Uncle BOB”)*

- S – Single-responsibility principle
- O – Open-closed principle
- L – Liskov substitution principle
- I – Interface segregation principle
- D – Dependency Inversion Principle

52

### 3.1. Nguyên lý một nhiệm vụ Single-responsibility Principle

- ❖ Mỗi lớp chỉ có một và duy nhất một lý do để thay đổi
- ❖ Cách nói khác: mỗi lớp chỉ có một nhiệm vụ duy nhất
- ❖ Thảo luận:
  - Mỗi lớp chỉ có một phương thức?
  - Tại sao mỗi lớp chỉ nên có một nhiệm vụ?

53

### Ví dụ 1

```
package test;
public class UserSettingService {
 public void
 changeEmail(User user) {
 if (checkAccess(user)) {
 //Grant option to change
 }
 }
 public boolean
 checkAccess(User user) {
 //Verify if the user is valid.
 }
}
```

54

### Ví dụ 1 - Mã nguồn sửa đổi

```
public class UserSettingService {
 public void changeEmail(User user) {
 if (SecurityService.checkAccess(user)) {
 //Grant option to change
 }
 }
}

public class SecurityService {
 public boolean checkAccess(User user) {
 //check the access.
 }
}
```

55

### Ví dụ 2

```
public class Employee{
 private String employeeId;
 private String name;
 private String address;
 private Date dateOfJoining;
 public boolean isPromotionDueThisYear(){
 //promotion logic implementation
 }
 public Double calcIncomeTaxForCurrentYear(){
 //income tax logic implementation
 }
 //Getters & Setters for all the private attributes
}
```

56

## Ví dụ 2 - Mã nguồn sửa đổi

```
public class HRPromotions{
 public boolean isPromotionDueThisYear(Employee emp){
 /*promotion logic implementation using the employee information
 passed*/
 }
}

public class FinITCalculations{
 public Double calcIncomeTaxForCurrentYear(Employee emp){
 /*income tax logic implementation using the employee information passed
 */
 }
}

public class Employee{
 private String employeeId;
 private String name;
 private String address;
 private Date dateOfJoining;
 //Getters & Setters for all the private attributes
}
```

57

## 3. 2. Nguyên lý đóng mở Open-closed Principle

- ❖ Các thực thể phần mềm (lớp, module, phương thức, ...) nên là MỞ cho các mở rộng nhưng ĐÓNG cho các sửa đổi (*open for extension, but closed for modification*)
  - “Open for extension”: một module (class) phải cung cấp các điểm mở rộng, cho phép thay đổi hành vi của nó
  - Closed for modification”: Mã nguồn của module không cần thay đổi để cài đặt sự mở rộng đó

58

58

## Ví dụ - HealthInsuranceSurveyor

```
public class HealthInsuranceSurveyor{
 public boolean isValidClaim(){
 System.out.println("Validating ...");
 /*Logic to validate health insurance claims*/
 return true;
 }
}
```

59

59

## ClaimApprovalManager

```
public class ClaimApprovalManager {
 public void processHealthClaim (HealthInsuranceSurveyor surveyor) {
 if(surveyor.isValidClaim()){
 System.out.println("Valid claim. Processing claim for approval....");
 }
 }
}
```

60

60

## ClaimApprovalManager

```
public class ClaimApprovalManager {
 public void processHealthClaim (HealthInsuranceSurveyor surveyor)
 {
 if(surveyor.isValidClaim()){
 System.out.println("Valid claim. Processing ...");
 }
 }
 public void processVehicleClaim (VehicleInsuranceSurveyor surveyor)
 {
 if(surveyor.isValidClaim()){
 System.out.println("Valid claim. Processing ...");
 }
 }
}
```

61

## Mã nguồn sửa đổi

```
public abstract class InsuranceSurveyor {
 public abstract boolean isValidClaim();
}

public class HealthInsuranceSurveyor extends InsuranceSurveyor{
 public boolean isValidClaim(){
 System.out.println("HealthInsuranceSurveyor: Validating claim...");
 /*Logic to validate health insurance claims*/
 return true;
 }
}

public class VehicleInsuranceSurveyor extends InsuranceSurveyor{
 public boolean isValidClaim(){
 System.out.println("VehicleInsuranceSurveyor: Validating claim...");
 /*Logic to validate vehicle insurance claims*/
 return true;
 }
}
```

62

## ClaimApprovalManager

```
public class ClaimApprovalManager {
 public void processClaim(InsuranceSurveyor
surveyor){
 if(surveyor.isValidClaim()){
 System.out.println("Valid claim. Processing ...");
 }
 }
}
```

63

## ClaimApprovalManagerTest

```
public class ClaimApprovalManagerTest {
 @Test
 public void testProcessClaim() throws Exception {
 HealthInsuranceSurveyor healthInsuranceSurveyor =
 new HealthInsuranceSurveyor();
 ClaimApprovalManager claim = new ClaimApprovalManager();
 claim1.processClaim(healthInsuranceSurveyor);

 VehicleInsuranceSurveyor vehicleInsuranceSurveyor =
 new VehicleInsuranceSurveyor();
 ClaimApprovalManager claim2 = new ClaimApprovalManager();
 claim2.processClaim(vehicleInsuranceSurveyor);
 }
}
```

64

## Ví dụ 2

```
public class Rectangle{
 private double length;
 private double width;
}

public class AreaCalculator{
 public double calculateRectangleArea(Rectangle
rectangle) {
 return rectangle.getLength() *rectangle.getWidth();
}
}
```



## Thêm lớp Circle

```
public class Circle{
 private double radius;
}

public class AreaCalculator{
 public double calculateRectangleArea(Rectangle
rectangle){
 return rectangle.getLength() *rectangle.getWidth();
}
 public double calculateCircleArea(Circle circle){
 return 3.14159*circle.getRadius()*circle.getRadius();
}
}
```



## Mã nguồn sửa đổi

```
public interface Shape{
 public double calculateArea();
}

public class Rectangle implements Shape{
 double length;
 double width;
 public double calculateArea(){
 return length * width;
}
}

public class Circle implements Shape{
 public double radius;
 public double calculateArea(){
 return 3.14159 *radius*radius;
}
}
```

## AreaCalculator

```
public class AreaCalculator{
 public double calculateShapeArea(Shape shape){
 return shape.calculateArea();
}
}
```

